

King
Saud University
College of Computer and Information Sciences
Department of Information Technology

Semester-2 1447H (Spring 2026)



CrashLens

IT 496: Graduation Project Report
Product Release-1

Prepared by

Aryam Almutairi, 444203968
Sarah Alruwayte, 444200758
Leen binmueqal, 444200885
Atheer Budie, 444200894

Supervised by
Dr. Luluah Alhusain

Second Semester 1447 H

Spring 2026



كلية علوم الحاسب والمعلومات
قسم تقنية المعلومات

Table of Contents

1	Introduction	6
1.1	The Problem.....	7
1.2	The Solution.....	8
1.3	Product	9
1.3.1	Product Vision	9
1.3.2	Product Roadmap.....	9
1.3.3	Objectives.....	10
1.3.4	Scope	12
1.3.5	Hardware/Software Tools and Cost.....	13
1.4	Scrum Team	14
1.4.1	Skill Set Requirements.....	14
1.4.2	Roles and Responsibilities	15
2	Background.....	16
2.1	Vehicle Damage Assessment	16
2.1.1	Traditional Vehicle Damage Assessment Process	17
2.1.2	Damage Classification Levels.....	18
2.1.3	Damage Assessment Report	18
2.2	Deep Learning.....	19
2.2.1	Convolutional Neural Networks (CNNs).....	20
2.2.2	YOLO (You Only Look Once)	20
2.2.3	Transfer Learning and Fine-Tuning	21
2.3	Computer Vision.....	22
2.3.1	Object Detection	22
2.3.2	Image Segmentation	23
2.3.3	Image Classification.....	24
2.3.4	Performance Evaluation.....	25
3	Literature Review	28
3.1	Vehicle Damage Detection	28
3.2	Damage Severity Estimation.....	30
3.3	Repair Cost Estimation:	32
3.4	Literature Findings.....	33
3.5	Competitive Product Analysis	34

4	System Description.....	40
4.1	Users	40
4.1.1	Individual Users (Vehicle Owners)	40
4.1.2	Administrators (Admin).....	40
4.2	Requirements Elicitation and Analysis.....	41
4.2.1	User Interactions.....	43
4.3	Product Backlog.....	44
4.4	Product Backlog Table.....	44
5	System Design	58
5.1	Architectural Diagram	58
5.2	Class Diagram.....	60
5.3	Component Level Design	61
5.4	Data Design.....	64
5.4.1	Data Models	64
5.4.2	Data Collection and Preparation.....	66
	Interface Design	68
6	System Implementation	74
6.1	Overview.....	74
6.2	Development tools and technologies	74
6.3	Challenges and Solutions.....	75
6.4	Vehicle Damage Detection	76
6.5	Functions.....	80
7	System Testing	85
7.1	Experimental Results	85
7.1.1	Discussion.....	87
7.2	User Acceptance Testing	88
7.2.1	Demographic Of Participants	88
7.2.2	Questionnaire Results	88
7.2.3	Discussion.....	91
8	Conclusion And Future Work.....	92
8.1	Summary	92
8.2	Global and Local Impact.....	92
8.3	Problems and Challenges Encountered.....	93
8.4	Limitations of the System	93

8.5	Main Contribution of the Project	94
8.6	Future Work	94
9	References	95
APPENDIX A: Interviews		98
APPENDIX B: User Acceptance Testing Questionnaire		105
APPENDIX C: UAT Questionnaire Responses		107

Table of Tables

Table 1: CrashLens Hardware/Software Tools and Cost Overview	13
Table 2: CrashLens Team Key Skill Requirements.....	14
Table 3: CrashLens Team Roles And Responsibilities Overview	15
Table 4:Car Damage Available datasets	30
Table 5:Damage severity Available datasets	32
Table 6: Feature comparisons	39
Table 7: Crashlens Definition of Ready.....	44
Table 8: Crashlens Product Backlog Table.....	44
Table 10: Experiments results summary	86

Table of figures

Figure 1: CrashLens Roadmap.....	9
Figure 2: Tagdeer online public report sample[11]	19
Figure 3: Neural network main structure [14].	20
Figure 4:YOLO Multi-Object Detection and Classification [16].	21
Figure 5: Performance Evaluation Metrics [21].	25
Figure 6: AI interface showing AI-based vehicle damage assessment [28].	35
Figure 7: Inspektlabs Workflow [29].....	36
Figure 8: RavinAI Workflow [29].	37

Figure 9:Tagdeer Workflow [8].....	38
Figure 10: Use case diagram of Crashlens.....	43
Figure 11: Crashlens MVC architecture	59
Figure 12:Class Diagram of CrashLens	60
Figure 13: General Pipeline of Crashlens	61
Figure 14: Vehicle Addition Flowchart	62
Figure 15: Submit Accident Case Flowchart	63
Figure 16: ER diagram.....	64
Figure 17:Non-Relational Data Model.	66
Figure 18:CarDD Dataset Samples.....	67
Figure 19:Annotation Sample.....	67
Figure 20: Mobile Application Site Map.....	68
Figure 21:Familiarity interface	69
Figure 22: Consistency interface	70
Figure 23: User Guidance interface	71
Figure 24: Feedback interface.....	72
Figure 25: Error prevention interface.	73

1 Introduction

Road traffic accidents represent a continuing challenge for transportation systems and insurance operations in the Kingdom of Saudi Arabia, as they frequently result in material vehicle damages that require accurate assessment before repair or insurance objection settlement. According to a national study on traffic accident analysis in Saudi Arabia, a total of 285,547 traffic accidents were recorded in the Kingdom during the year 1439H [1], reflecting the substantial volume of incidents that lead to vehicle damage and associated financial losses.

Commented [41]: add citation for this. number

In Saudi Arabia, traffic accidents represent a major economic and administrative concern due to their impact on vehicle repair costs, insurance objections, and service efficiency. National studies emphasize that traffic accidents lead to significant material losses, including vehicle damage and infrastructure costs, which place pressure on insurance companies and official assessment authorities [2], [3]. As the number of vehicles continues to increase, traditional damage assessment methods primarily based on manual inspections are becoming increasingly time-consuming, resource-intensive, and prone to subjectivity [4].

Recent advances in artificial intelligence, particularly in computer vision and deep learning, have demonstrated strong potential in automating vehicle damage detection and severity estimation from images. Research in this field shows that AI-based systems can analyze vehicle images to identify damaged parts, classify damage severity, and support insurance-related decision-making with higher consistency and efficiency compared to purely manual approaches. Such technologies enable faster preliminary assessments, reduce operational delays, and support scalable insurance services.

From a national perspective, the adoption of intelligent digital solutions aligns with the objectives of Saudi Vision 2030, which emphasizes digital transformation, efficiency improvement, and the use of advanced technologies to enhance service quality across sectors, including transportation and insurance [5]. Implementing AI-powered damage assessment solutions contributes to improving operational efficiency, supporting data-driven decision-making, and reducing reliance on manual processes.

This graduation project proposes CrashLens, an AI-powered mobile application that allows vehicle owners to submit images of damaged vehicles and receive an initial estimation of damage severity and approximate repair cost. The system aims to support users by providing a convenient and consistent assessment while offering insurance-related stakeholders structured and verifiable assessment reports. This document presents the problem addressed by the project, the proposed solution, product vision, roadmap, objectives, scope, required tools, the Scrum team structure, the system design, implementation, and evaluation of Release 1 of the CrashLens application, including the developed features, system architecture, and experimental results.

1.1 The Problem

After a road traffic accident, vehicle owners are typically required to assess the damage in order to proceed with repair or insurance objections. This process often involves visiting authorized inspection centers and relying on human assessors to evaluate the extent of the damage. This dependency on in-person inspection makes the process time-consuming and inefficient, particularly during peak periods when assessment centers must handle a high volume of insurance objections [4]. In addition to delays, manual vehicle damage assessment is inherently subjective. Different inspectors may provide varying evaluations for the same damage, resulting in inconsistent severity judgments and repair cost estimates. Such inconsistencies often lead to disputes between vehicle owners and insurance providers, reducing transparency and increasing the overall processing time of insurance objections.

The practical impact of these limitations can be observed in common real-world accident assessment scenarios. For example, after a minor traffic accident, a vehicle owner may need to take time off work to visit an authorized inspection center, wait for an assessor to review the damage, and then wait again for an official cost estimate. In some cases, the assessment may differ from another evaluator's judgment, leading to further delays, reassessments, or disputes with the insurance company.

Despite the widespread availability of digital images and advances in computing technologies, there is still limited adoption of intelligent systems capable of providing initial, data-

driven damage assessments based on visual evidence. The absence of such systems represents a missed opportunity to reduce processing time, improve assessment consistency, and enhance user experience. This project focuses specifically on addressing this gap by providing an automated, image-based approach for vehicle damage severity and cost estimation, supporting but not replacing official inspection procedures.

1.2 The Solution

To address the limitations of manual vehicle damage assessment, this project proposes CrashLens, an AI-powered mobile application designed to provide a vehicle damage evaluation using images uploaded by users. The system enables vehicle owners to capture and submit guided images of damaged vehicles along with basic vehicle information, such as vehicle model and manufacturing year, through a user-friendly mobile interface.

The proposed solution follows an automated image-based assessment approach that combines computer vision and machine learning techniques to analyze visible vehicle damage. After image submission, the system processes the captured vehicle images, identifies damaged regions, estimates the severity of the detected damage, and generates a repair cost estimate based on the assessment results and vehicle-related information.

To improve the quality and consistency of the analysis process, the application guides users during image capture to ensure sufficient image clarity and coverage of the damaged areas. The generated assessment is intended to support faster evaluations and improve consistency compared with purely manual inspection processes.

Overall, the proposed solution aims to improve accessibility, reduce insurance claim processing delays, and provide users with a more efficient and transparent vehicle damage assessment experience while supporting, rather than replacing, official inspection procedures.

1.3 Product

1.3.1 Product Vision

For vehicle owners who need an assessment of vehicle damage after an accident,

CrashLens is an AI-powered mobile based system that detects damage, estimates its severity and approximates repair costs from uploaded vehicle images.

Unlike traditional manual inspection methods, which are often time-consuming and subjective, CrashLens provides a rapid, accurate, data-driven assessment.

1.3.2 Product Roadmap

The development of the CrashLens system follows an incremental agile approach using the Scrum methodology. The project is implemented over two academic semesters through two major releases, where each release consists of multiple sprints that deliver usable system increments. Figure 1 illustrates the development roadmap of the project.

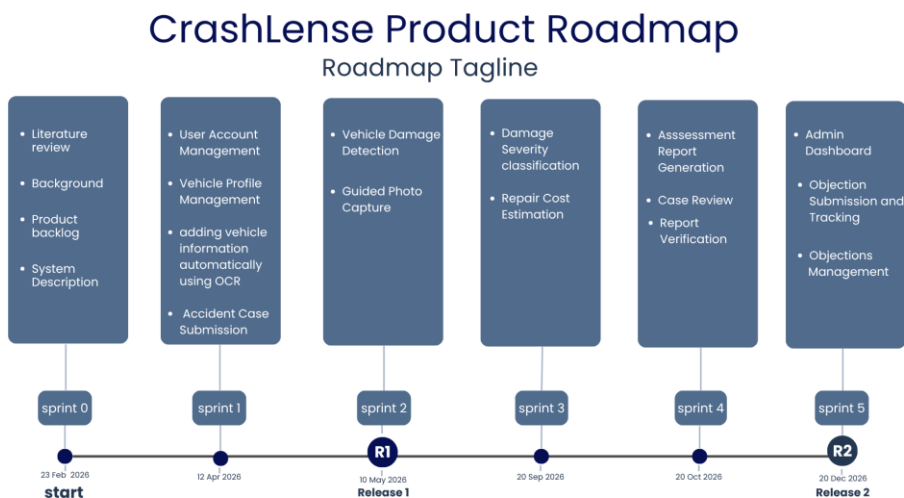


Figure 1: CrashLens Roadmap

1.3.3 Objectives

The objectives of this graduation project are defined on three levels: **product**, **project**, and **learning**.

Product Objectives (Customer focus – value)

CrashLens is an AI-powered mobile application designed to provide vehicle owners with a fast and consistent damage assessment after an accident. The product objectives describe the value that CrashLens provides to its intended users, including vehicle owners and insurance companies.

By the end of the project, CrashLens aims to:

1. To enable vehicle owners to register, manage their accounts, and maintain vehicle information within the system.
2. To allow users to register accident cases and submit detailed vehicle damage information, including accident details and relevant vehicle images.
3. To develop automated damage assessment capabilities, including damage detection, severity estimation, and repair cost estimation
4. To generate automated assessment reports summarizing accident details, detected damage, severity levels, and estimated repair costs.
5. To provide report verification functionality that enables third parties to validate the authenticity and status of generated assessment reports.
6. To develop an administrative dashboard that allows authorized administrators to review accident cases and approve or adjust damage assessments.
7. To support objections management by enabling users to submit objections to assessment results and allowing administrators to review and respond to them through the system.

Project Objectives (Solution focus – plan)

The project objectives describe the technical and developmental activities required to design, implement, and deliver the CrashLens system across two academic semesters.

1. Conduct a background study and literature review to understand vehicle damage assessment processes and existing AI-based approaches.
2. Obtain and prepare the CarDD dataset for vehicle damage assessment, including data cleaning, organization, augmentation, and splitting.
3. Design the overall system architecture, including frontend, backend, database, and model interaction.
4. Develop the backend infrastructure and database structure required to store user, vehicle, accident, and assessment data.
5. Develop and train models to detect vehicle damage, estimate damage severity, and predict repair costs.
6. Evaluate and improve model performance through testing, experimentation, and refinement.
7. Integrate the trained models into the system.
8. Perform system testing and refinement to ensure correctness, reliability, and usability.
9. Prepare final documentation and project deliverables.

Learning Objectives (Student focus – skills gained)

The learning objectives describe the knowledge and skills the project team aims to acquire beyond the standard IT curriculum through the development of the CrashLens system. By completing this project, the team will:

1. Gain practical experience in applying computer vision techniques to real-world image-based vehicle damage assessment problems.
2. Learn how to design and implement multiple machine learning models for related tasks, including damage detection, damage severity estimation, and repair cost estimation.

3. Develop hands-on experience with the complete machine learning workflow, including data collection, data preparation, preprocessing, augmentation, training, evaluation, and model refinement.
4. Learn practical transfer learning and fine-tuning techniques using convolutional neural networks (CNNs) for image-based classification and detection tasks.
5. Strengthen skills in backend and system integration, including connecting machine learning models with application services, databases, and user-facing components.
6. Gain experience in building and managing a data-driven software system using Agile/Scrum practices, including sprint planning, backlog management, teamwork, and iterative delivery across multiple releases.

1.3.4 Scope

This project focuses on the development of an AI-powered mobile-based system that provides a, image-based assessment of vehicle damage after road accidents. CrashLens is intended for vehicle owners aged 18 and above, as well as insurance companies who benefit from consistent and verifiable assessment reports

The system enables users to create accounts, register vehicles, and submit accident objections by uploading vehicle images, along with basic vehicle and accident information. Based on the provided data, the system generates an assessment that includes damage type detection (e.g., dent, scratch, crack, glass shatter, lamp broken, and tire flat), severity classification (e.g., minor, moderate, or severe), and an approximate repair cost estimate.

The scope of this project is limited to the assessment of externally visible vehicle damage based on images. The system is developed exclusively for Android platforms and supports the Arabic language only. Integration with official insurance systems, and detection of internal or mechanical vehicle faults are out of scope.

1.3.5 Hardware/Software Tools and Cost

Table 1 displays the hardware/software tools and cost for the CrashLens platform.

Table 1: CrashLens Hardware/Software Tools and Cost Overview

Hardware Tools	
Name and Description	Cost
Laptops for development	Already available
Mobile for Testing	Already available
Software Tools	
Name and Description	Cost
Flutter (mobile application development framework)	Free
Figma (UI/UX Design tool)	Free
Firebase (authentication, database, and storage)	Free
Google Colab (model development and experimentation)	Paid (Google Colab Pro) 82.51 SAR
Visual Studio Code (VS Code) – IDE	Free
OpenCV (image processing and computer vision)	Free
Programming languages like (Dart, Python)	Free
TensorFlow or PyTorch (deep learning framework)	Free
Kaggle datasets (training and evaluation data)	Free
Jira (Scrum project management)	Free
GitHub (version control and collaboration)	Free

1.4 Scrum Team

1.4.1 Skill Set Requirements

Table 2 displays the key skills requirement for the team of the CrashLens platform.

Table 2: CrashLens Team Key Skill Requirements.

Technical Skill Required	The team's current level, Learning plan
Mobile App Development (Flutter/Dart)	The team's level is basic; the team has basic knowledge in developing cross-platform mobile applications using Flutter. To strengthen proficiency, team members will complete targeted Flutter tutorials and implement core application screens during the first development sprint.
Backend (Python +FastAPI)	The team's level is basic, the team has introductory experience in backend. This skill will be developed further by implementing simple backend functions early in the project and gradually extending backend functionality across later sprints. Suggested learning resources: FastAPI official documentation and "Python Backend Development" tutorials on YouTube.
Database (Firebase)	The team's level is basic, the team has basic familiarity with Firebase services for authentication, database management, and cloud storage. The team will enhance this knowledge through practical implementation, starting with user authentication and image storage.
Machine Learning: Computer Vision + YOLOv8	The team has beginner-level experience in machine learning and computer vision. During early sprints the focus will be on understanding the selected dataset and conducting baseline experiments. Later on, the team will develop and fine-tune YOLOv8 models for vehicle damage detection tasks. Suggested learning resources: taking the machine learning and Computer Vision course and using our self-learning skills.
Data Preparation & Preprocessing (resize, normalize, augmentation)	The team has intermediate experience in preparing image data. This skill will be strengthened by applying consistent preprocessing pipelines and experimenting to improve model robustness.
Software Engineering Tools (Git/GitHub, Jira, versioning)	The team has intermediate experience in collaborative software development tools such as GitHub and Jira. These tools will be used throughout the project to manage sprint tasks, maintain version control, and support Scrum-based teamwork.

Learning

We deepened our understanding by conducting a background study and reviewing literature on vehicle damage assessment. Additionally, we conducted in-depth research to explore competitive applications such as Taqdeer, and ensure our approach effectively meets the needs of vehicle owners in Saudi Arabia while minimizing assessment costs and eliminating the need for physical visits. Furthermore, a team members completed the "Introduction to Computer Vision and Image Processing" course on Coursera, which strengthened the team's understanding of image processing, machine learning, and object detection concepts [6].

1.4.2 Roles and Responsibilities

Table 3 displays the roles and responsibilities for the team of the CrashLens platform.

Table 3: CrashLens Team Roles And Responsibilities Overview

Scrum Team	
Product Owner:	Dr. Luluah Alhusain
Developers:	Atheer Budie Aryam Almutairi Leen binmueqal Sarah Alruwayte
Scrum Master (SM):	Dr. Luluah Alhusain
Stakeholders:	Vehicle owners Insurance companies

2 Background

This chapter presents the domain and technical foundations required to understand the CrashLens project. It introduces vehicle damage assessment and explains the core technologies that enable automated, image-based damage evaluation. Since CrashLens relies on analyzing user-captured vehicle images, the background focuses on computer vision techniques for damage detection, severity estimation, and repair cost approximation, along with suitable performance metrics for evaluating system effectiveness.

2.1 Vehicle Damage Assessment

Vehicle damage assessment is a structured technical process conducted after road traffic accidents to determine the type, location, and severity of vehicle damage. The primary objective of this process is to identify the required repair actions and estimate the associated financial cost. Damage assessment represents a critical stage in post-accident procedures, as it forms the foundation for insurance objection settlement, repair planning, and compensation decisions [7].

The accuracy and reliability of a vehicle damage assessment depend on several factors, including the expertise of the assessor, the quality of available documentation, and adherence to standardized evaluation procedures. An incomplete or inconsistent assessment may lead to financial disputes between vehicle owners, repair workshops, and insurance providers. Therefore, damage assessment systems are typically governed by professional and regulatory standards to ensure fairness and transparency.

In Saudi Arabia, vehicle damage assessment activities are regulated under the supervision of accredited valuation authorities, such as the Saudi Authority for Accredited Valuers (Taqeem). Certified assessors operate through authorized centers that utilize unified electronic systems, such as Taqdeer [8], to issue standardized accident assessment reports. These reports serve as the official technical reference for determining repair feasibility and calculating compensation amounts.

2.1.1 Traditional Vehicle Damage Assessment Process

The traditional damage assessment process is primarily manual and requires physical inspection of the vehicle. After an accident occurs, the vehicle owner must obtain an official accident report, such as a Najm report, and then visit an authorized assessment center. A certified assessor examines the damaged vehicle in person, documents the affected parts, captures photographic evidence, and evaluates the extent of the damage.

During this inspection, the assessor identifies damaged components, determines whether each part requires repair or replacement, and classifies the severity of the damage. The total repair cost typically consists of two main components: spare part cost and labor cost. Spare part cost depends on the vehicle's make, model, and whether original manufacturer parts or alternative parts are used. Labor cost reflects the time and technical effort required to repair or replace the damaged components.

Additional factors may influence the final estimate, such as paint type, regional labor rates, and workshop standards. For example, vehicles with metallic or specialized paint finishes may require additional materials and labor procedures, increasing the overall repair expense. Because these factors vary between workshops and regions, cost estimates are often expressed as a range rather than a fixed value.

Based on these findings, a structured assessment report is prepared, including technical descriptions and estimated repair costs. This report is then submitted to the relevant insurance company to initiate compensation procedures.

Although this process ensures compliance with regulatory standards and professional guidelines, it is time-consuming and dependent on human judgment. Scheduling physical inspections, documenting damage manually, and waiting for official approval may result in delays, particularly during periods of high accident volume. Furthermore, differences in assessor experience or workshop pricing practices may lead to variations in cost estimation.

2.1.2 Damage Classification Levels

To maintain consistency and standardization, vehicle damages are typically categorized into defined severity levels. These levels guide repair decisions and influence the final cost estimation [9], [10].

Minor damage generally refers to superficial or cosmetic impacts, such as small dents, light scratches, or minor bumper deformation. These damages often require limited repair procedures such as repainting, polishing, or minor part adjustments.

Moderate damage involves more significant structural or panel impact, including large dents in doors or fenders, partially broken bumpers, or noticeable deformation of exterior components. Such cases may require part replacement, welding, or more extensive repair work.

Severe damage represents critical structural or functional impairment. Examples include bent chassis components, damaged suspension systems, deployed airbags, or major frame distortion. Severe damage significantly increases repair complexity and cost, and in some cases may result in a total-loss evaluation.

Severity classification plays an essential role in determining the technical repair strategy and the financial value of the estimated damage.

2.1.3 Damage Assessment Report

Following inspection and cost evaluation, a structured damage assessment report is generated. Based on publicly available samples of official Taqdeer reports [9] such documents typically include the accident reference number, vehicle identification details, owner information, description of accident circumstances, list of damaged components, recommended repair or replacement actions, and an estimated cost breakdown, as shown in Figure 2. The report also contains assessor identification details and official validation to ensure authenticity.

The damage assessment report serves as crucial legal and financial function. Insurance companies rely on the report to determine compensation amounts and approve repair procedures.

Deep learning models rely on multi-layer neural networks that progressively learn hierarchical representations. In image-based tasks, early layers typically detect low-level features such as edges and textures, while deeper layers learn higher-level semantic patterns such as object parts and structures [13].

2.2.1 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are specialized deep learning architectures designed for image processing. As illustrated in Figure 3 CNNs consist of convolutional layers, pooling layers, and fully connected layers.

- Convolutional layers extract spatial features using learned filters.
- Pooling layers reduce spatial dimensions while preserving important features.
- Fully connected layers perform classification based on extracted representations.

CNNs form the backbone of most modern computer vision systems, including classification, detection, and segmentation tasks. Their ability to handle spatial hierarchies makes them particularly effective for analyzing complex real-world images.

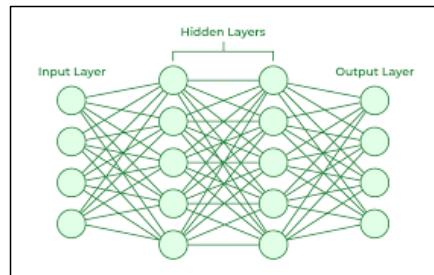


Figure 3: Neural network main structure [14].

2.2.2 YOLO (You Only Look Once)

YOLO is a single-stage object detection architecture designed for high-speed inference while maintaining competitive accuracy. Unlike two-stage detectors that first generate region

proposals and then classify them, YOLO performs detection in a single forward pass through the network [15].

Key characteristics of YOLO include:

- Real-time detection capability.
- End-to-end training.
- Simultaneous prediction of bounding boxes and class probabilities.

YOLO variants (e.g., YOLOv5, YOLOv7, YOLOv8) introduce architectural improvements for better accuracy and computational efficiency. Due to its balance between performance and speed, YOLO has become widely adopted in applied computer vision systems, as shown in Figure 4.

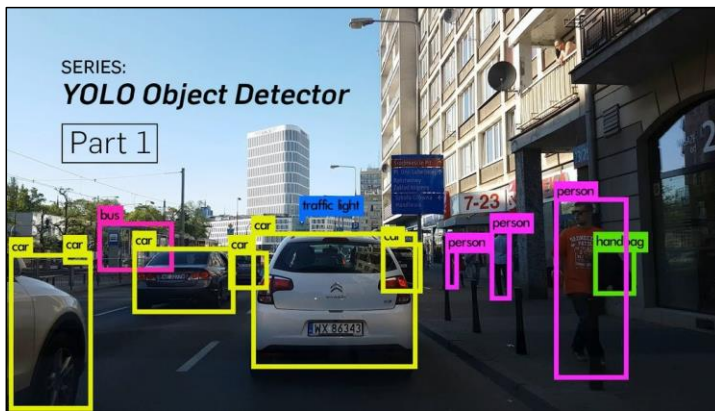


Figure 4: YOLO Multi-Object Detection and Classification [16].

2.2.3 Transfer Learning and Fine-Tuning

Transfer learning is a training strategy where a model pre-trained on a large dataset is adapted to a new, domain-specific task. Since collecting large-scale labeled vehicle damage

datasets is challenging, transfer learning significantly improves model performance and reduces training time [17].

Fine-tuning is a critical step in transfer learning. It involves:

- Maintaining early convolutional layers that capture generic visual features.
- Re-training later layers to adapt to domain-specific patterns.

Fine-tuning improves generalization, reduces overfitting, and enhances model robustness in real-world conditions. Due to limited domain datasets in automotive damage analysis, fine-tuning is considered a crucial methodological component [18].

2.3 Computer Vision

Computer Vision is a subfield of artificial intelligence (AI) that equips machines with the capability to process, analyze, and interpret visual inputs, such as digital images and videos. By utilizing machine learning, computer vision enables computers and systems to derive meaningful information from visual data. Within CrashLens, computer vision is used to interpret photographs of damaged vehicles captured via mobile devices [19], [20].

2.3.1 Object Detection

Object detection is a multifaceted computer vision task that functions by extracting both the semantic identity and the precise spatial coordinates of multiple entities within a single digital frame. At its core, the process is defined by the generation of bounding boxes, which are represented by a four-coordinate vector typically (x, y, w, h) defining the centre, width, and height of the detected object. Each box is inextricably linked to a class label and a confidence score, a scalar value representing the model's architectural certainty that the identified region contains a valid object of interest.

In practical application, such as vehicle damage analysis, object detection serves as the primary diagnostic tool for localizing affected components. Rather than evaluating a scene globally, the model identifies specific regions of interest, such as a "fractured headlight" or "dent," by isolating those features from the background pixels. This allows for a granular analysis where

a system can distinguish between a single point of impact and multiple, disparate areas of damage on a single vehicle, providing a mapped "hit-list" of required repairs.

2.3.2 Image Segmentation

Image represents the most granular tier of computer vision, transcending the spatial limitations of object detection. While detection identifies an object's presence within a rectangular coordinate system, segmentation performs pixel-level classification, assigning a specific semantic label to every individual pixel in the image. This transition from "coarse localization" (boxes) to "fine-grained partitioning" (masks) allows for a mathematical understanding of an object's exact geometry and contours.

In the context of vehicle damage analysis, this level of detail is transformative. By identifying the specific pixels that represent "deformed metal" or "shattered glass," segmentation models can calculate the exact surface area and boundaries of a dent. This provides a significantly more accurate metric for repair cost estimation than a bounding box, which inevitably includes "noise" pixels from the undamaged background surrounding the impact site.

The architectural methodology for achieving this pixel-perfect accuracy generally splits into two primary functional approaches:

- **Semantic Segmentation:** This approach treats the image as a collection of categories. Every pixel belonging to the "car" class is colored the same, regardless of how many individual cars are in the frame. It is essentially a dense classification task that focuses on the "what" and "where" across the entire scene.
- **Instance Segmentation:** This approach combines the strengths of object detection and semantic segmentation. It not only identifies the class of each pixel but also differentiates between separate objects of the same class. In a multi-car accident, instance segmentation would assign a unique ID to "Car A" and "Car B," ensuring their overlapping boundaries are clearly defined.

To process these complex spatial relationships, segmentation models typically utilize Encoder-Decoder architectures.

- **U-Net:** This symmetric architecture uses an "encoder" to capture the context of the image and a "decoder" to enable precise localization. It is famous for its "skip connections" that recover fine-grained spatial information lost during downsampling.
- **Mask R-CNN:** An evolutionary step above detection, Mask R-CNN adds a third branch to the standard Faster R-CNN architecture. While two branches predict the class and the bounding box, the third branch predicts a binary mask for each detected object, effectively merging detection and segmentation into a single high-performance pipeline.

2.3.3 Image Classification

Image Classification serves as the foundational pillar of computer vision, tasked with assigning a definitive label to an entire image based on its global visual characteristics. While rudimentary classification identifies the subject of a photo, its application within the automotive sector has evolved into a sophisticated automated damage assessment pipeline. This end-to-end system begins with image acquisition, where multi-angle visual data is captured, followed by damage detection to isolate structural anomalies.

The core of this process is severity estimation, which utilizes classification algorithms to provide standardized evaluation outcomes categorizing damage into discrete levels such as "Minor," "Moderate," or "Severe." This classification is critical because it informs the final stage of the pipeline: repair cost approximation, where the system translates visual severity into a data-driven financial estimate.

To handle the high variance in vehicle surfaces, such as reflections and complex geometries, the industry relies on advanced Deep Convolutional Neural Network (CNN) architectures:

- **ResNet:** Employs residual learning to overcome the degradation problem in very deep networks, ensuring stable feature extraction.
- **VGG:** Utilizes a simplified, deep architecture with small convolutional filters to capture fine-grained spatial hierarchies.
- **EfficientNet:** Optimizes performance by scaling depth, width, and resolution simultaneously, providing high-accuracy results with maximum computational efficiency. Together, these models enable a seamless transition from raw visual input to precise, automated diagnostic reports.

2.3.4 Performance Evaluation

Standard evaluation metrics are used to assess the performance of the proposed computer vision models in detecting vehicle damage and affected regions. These metrics help measure detection accuracy, completeness, and localization quality, ensuring reliable and objective evaluation of the system [21], [22]. As illustrated in Figure 5, the evaluation process is based on four key classification outcomes:

- **True Positives (TP)** :occur when the model correctly identifies and localizes an object of interest.
- **True Negatives (TN)**: occur when the model correctly identifies that an object of interest is not present.
- **False Positives (FP)**: occur when the model incorrectly identifies an object of interest where there is none.
- **False Negatives (FN)**: occur when the model fails to identify an object of interest that is present.

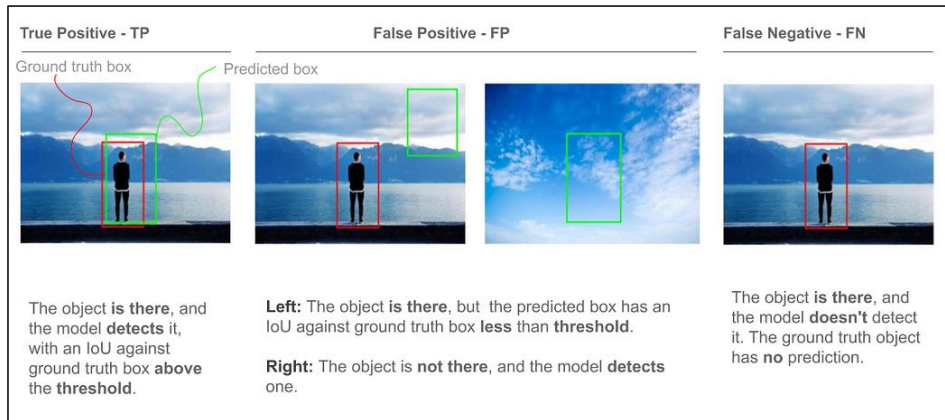


Figure 5: Performance Evaluation Metrics [21].

Detection Performance Metrics:

Object detection models are evaluated based on both classification accuracy and localization quality.

- **Precision(P):** is a metric used to measure the accuracy of the positive predictions made by an object detection model.

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall:** is a metric used to measure the completeness of the object detection model in identifying all relevant objects within a dataset.

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

A high recall value means that the model successfully identifies most of the relevant objects, with few missed detections

- **Intersection over Union (IoU):** is a metric used to evaluate the accuracy of object detection models. It measures the overlap between the predicted bounding box and the ground truth bounding box. The IoU is defined as the ratio of the area of intersection to the area of union of the two bounding boxes.

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

- **Mean Average Precision (mAP):** is the meaning (average) of AP values across different classes and/or different IoU thresholds. So, the mAP is the meaning of the AP values across all classes in the dataset.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N A P_i$$

where N is the number of classes and AP_i is the average precision for the i-th class.

Performance Metrics for Severity Classification:

- **Accuracy:** evaluates how well a machine learning model performs. It represents the percentage of correct predictions made by the model.

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision and Recall**

Used when class imbalance exists between severity levels.

- **F1 Score:** is a single metric that combines precision and recall providing a balanced measure of a model's performance.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F1 Score ranges from 0 to 1, where 1 indicates perfect precision and recall, and 0 indicates the worst performance.

- **Confusion Matrix:** provides detailed insight into misclassification patterns between severity categories, helping evaluate robustness and bias.

3 Literature Review

This chapter provides a structured review of existing literature, categorized into three core domains: damage detection, severity estimation, and repair cost estimation, focusing on the methodologies that form the foundation of this field.

3.1 Vehicle Damage Detection

In the primary stage of identifying car damage, Zhang et al. (2020) [23] developed a specialized approach using an improved Mask R-CNN algorithm designed to detect small scale damages such as scratches and subtle dents. Their methodology involved refining the Feature Pyramid Network (FPN) to better capture multi scale features and implementing RoIAlign to ensure precise pixel-level edge detection instead of using standard bounding boxes. The model was trained on a self-collected dataset of 2,000 real-world accident images annotated with high-precision masks using the Labelme tool. The results demonstrated a detection accuracy of 96.68%, proving that precise segmentation is a critical requirement for accurate damage area calculation.

Furthermore, research by Jin et al. (2023) [24] explored the efficiency of the YOLO architecture, which utilizes a single stage detection methodology to prioritize processing speed for mobile environments. The model was trained on large scale datasets, such as CarDD, featuring diverse vehicle types under varied lighting and camera angles to simulate real-world road accidents. The results showed that YOLO excels in real time inference speed while maintaining high precision.

Additionally, Hasan et al. (2025) [25] conducted a systematic literature review to provide a comprehensive overview of AI-based research in vehicle damage detection, with a particular focus on deep learning approaches. The study analyzes multiple research works to evaluate different methodologies and their performance, highlighting the trade-offs between detection speed and accuracy. Their findings indicate that YOLO-based models are well-suited for real-time applications due to their high processing speed, while instance segmentation methods remain more appropriate for insurance-grade assessments that require precise damage localization. Furthermore, the study emphasizes the importance of integrating image

enhancement techniques to address real-world challenges such as poor lighting and environmental conditions.

Datasets for Damage Detection

Several publicly available datasets were explored and evaluated to identify suitable training data for the CrashLens damage detection model.

CarDD: contains high-resolution vehicle damage images with 9,000+ annotated instances. It provides both bounding boxes and segmentation masks. Annotations follow insurance-oriented rules aligned with real repair and objection processing workflows. Supports damage classification, object detection, instance segmentation, and salient object detection.

COCO: A small dataset of approximately 80 vehicle damage images annotated in COCO JSON format with bounding boxes. Supports object detection only. Lightweight and easy to download, suitable only for early pipeline testing and debugging.

Coco annotated car damage detection dataset: contains 391 images and 657 annotated damage instances across 7 classes in COCO format. Supports object detection and instance segmentation. Category definitions differ from CarDD (e.g., includes Dislocation but lacks Tire Flat and Lamp Broken).

Vehicle: One of the largest publicly available vehicle damage datasets with 13,945 high resolution real accident images and 32,000+ annotated instances. Uses VIA polygon segmentation format. Dataset is imbalanced, with scratches dominating (12,267 instances). Annotation format requires conversion to COCO/YOLO before use.

Commented [43]: better here as text, sice too much description

As presented in Table 4, each dataset was analyzed based on its size, damage classes, image characteristics, annotation format, and source availability. The datasets differ in scale, image resolution, annotation structure, and damage taxonomy, making a structured comparison necessary before selecting the most appropriate dataset for training and evaluation.

The Decision column, highlighted in Table 4, represents the project team’s evaluation and selection rationale rather than an inherent property of the datasets themselves. This separation was intentionally maintained to distinguish objective dataset information from the project’s assessment criteria. Based on this comparative analysis, CarDD was selected due to its high-

resolution images, large-scale annotations, realistic damage categories aligned with insurance repair workflows, and support for both bounding box detection and instance segmentation.

Table 4: Car Damage Available datasets

Dataset	Size	Classes	Link	Decision
carDD	Train:2816 Test: 374 Val: 810	1. Dent 2. Scratch 3. Crack 4. Glass Shatter 5. Tire Flat 6. Lamp Broken	link	Accepted – Large-scale, high-resolution, well-annotated, realistic damage categories.
coco	Train: 59 Val: 11	1. Headlamp 2. Front_Bumpe 3. Hood 4. Door 5. Rear_Bumper	link	Rejected – Very small dataset size, insufficient for reliable training.
Coco annotated car damage detection dataset	Train: 421 Val: 182	1. Dent 2. Large_dent 3. Scratch_or_spot 4. Tear 5. Large_tear_or_damage 6. Shatter 7. Dislocation	link	Rejected – Small dataset size, inconsistent damage taxonomy.
Vehide	Train: 11600 Val: 2324	1. Paint damage 2. Dent deformation 3. Tears 4. Missing or broken parts 5. Sever deformation 6. Holes or punctures 7. Broken glass	link	Rejected – Class imbalance, Vietnamese annotations, preprocessing complexity.

Commented [سا4]: تعديل

Commented [45]: this column should be rewritten. Accepted or rejected and this add short reasons, such as: High resolution, annotated. or small size..

3.2 Damage Severity Estimation

Damage Severity involves classifying the intensity of the damage into categories such as minor, moderate, or severe. Qaddour and Siddiqi (2023) [10] introduced a robust estimator utilizing the Inception-ResNetV2 architecture. Their methodology relied on Transfer Learning to train the hybrid model which combines residual connections to prevent data loss with inception modules for complex feature extraction to distinguish between subtle damage

intensities across different car parts. The study utilized two custom-constructed datasets: the first contained images of cars alongside various objects, while the second focused specifically on damaged and undamaged vehicles. The damage in the second dataset was categorized into significant, moderate, and minor severity levels. Data augmentation techniques such as flipping, zooming, shifting, and random rotation were applied to combat overfitting during training. The model achieved a high accuracy rate of 92%, validating that deep learning can effectively reduce the subjectivity associated with human assessment.

Complementing this, Mallios et al. (2023) [26] addressed the challenges of "In-The-Wild" images captured by customers in unconstrained environments. They implemented a validation pipeline to ensure image quality before the analysis began. Their technical methodology focused on calculating a "Damage Ratio" by computing the Intersection over Union (IoU) between the damage mask and the vehicle part mask. This quantitative approach allows the system to determine the severity based on the actual percentage of deformation. Their results proved that this method provides a more objective standard for deciding whether a part requires repair or replacement compared to traditional visual estimations. The model was trained and evaluated on a large dataset of historical insurance objections with known outcomes, with all images captured in real-world conditions using mobile-phone hardware. Damage in the dataset was labelled across five different types, and segmentation masks were created for all visible damage as well as the vehicle part boundaries.

Datasets for Damage Severity

Several publicly available datasets were reviewed to explore potential data sources for the damage severity estimation component of the CrashLens system. As summarized in Table 5, the datasets were compared based on their size, severity categories, and overall suitability for estimating the extent of vehicle damage.

Table 5: Damage severity Available datasets

Dataset	Size	Classes	Link
Car Damage Severity Dataset	- Train: Minor: 452 Moderate: 463 Severe: 468 - Val: Minor: 82 Moderate: 75 Severe: 91 Total = 1631	Minor Moderate Severe	Link
Car Damage Detection (for severity)	- Train: Minor: 1076 Moderate: 924 Severe: 945 - Val: Minor: 82 Moderate: 157 Severe: 151 Total = 3335	Minor Moderate Severe	Link

3.3 Repair Cost Estimation:

Repair Cost Estimation involves converting visual findings into standardized financial valuations. Elbhrawy et al. (2024) [27] developed the Cost Estimation System (CES) that does not predict price directly via AI but uses a Rule-Based Calculation Engine that integrates AI-detected damage (part + severity) with a pricing database and structured customer data (vehicle make, model, year of manufacture) to calculate the total using explicit formulas.

The following variables are used in the calculation process:

C_p = Cost of Parts , C_l = Labor Cost , C_o = Overhead Cost

C_{pr} = Profit Cost , C_t = Total Repair Cost

R = Hourly Labor Rate , T = Estimated Repair Time

O = Overhead Percentage , P = Profit Percentage

The repair cost is then calculated using the following equations:

$$C_p = \sum(\text{price of each part})$$

$$C_l = R \times T$$

$$C_o = O \times C_l$$

$$C_{pr} = P \times (C_p + C_l + C_o)$$

$$C_t = C_p + C_l + C_o + C_{pr}$$

Relying on the visual inputs from the YOLOv8 model (78.50% precision) and Transformer-based body part detection, the system retrieves correct part prices from a database and calculates total cost through transparent mathematical formulas rather than black-box machine learning predictions. The CES successfully generated comprehensive itemized repair reports including parts, labor, and painting costs, providing satisfactory performance for automating objections workflows while maintaining full traceability and regulatory compliance. This approach offers explainability advantages over predictive models, allowing insurance adjusters and customers to understand exactly how repair costs are calculated, though it requires maintaining accurate pricing databases and updating labor rates regularly.

3.4 Literature Findings

Overall, the reviewed studies demonstrate that automated vehicle damage assessment is most effective when implemented as a multi-stage pipeline rather than a single predictive model. Detection models such as Mask R-CNN provide highly accurate damage detection suitable for documentation level analysis, while lightweight detectors like YOLO offer practical real-time performance required for mobile applications. For severity estimation, deep convolutional networks using transfer learning significantly provide consistent classification outcomes. Finally, cost estimation research indicates that rule-based calculation frameworks combined with AI outputs offer higher transparency and reliability compared to purely predictive pricing models

Based on the reviewed literature, CarDD was selected as the dataset for the CrashLens damage detection model, as it provides high-resolution images, instance-level annotations, and damage categories aligned with insurance repair workflows.

3.5 Competitive Product Analysis

Automated vehicle damage assessment has been addressed by both global artificial intelligence companies and locally regulated assessment services. To position the proposed system within the current technological and operational landscape, four representative solutions were analyzed: Tractable AI, Inspektlabs, Ravin AI, and Taqdeer. These systems differ in target users, operational models, and technological approaches, providing a comprehensive basis for comparison.

Tractable AI



Tractable

Tractable AI [28] is a global technology company specializing in AI-driven objections automation for insurance providers. The platform primarily serves insurance companies rather than individual vehicle owners. Its core value proposition centers on accelerating objections processing through automated visual assessment as illustrated in Figure 6.

Publicly available information indicates that Tractable employs deep learning-based computer vision models to analyze vehicle images and generate damage assessments. The system performs image-based damage detection and produces repair cost estimates integrated into insurer workflows. While specific datasets used for model training are not publicly disclosed, Tractable reports training its models on large-scale proprietary insurance datasets to achieve enterprise-level performance. The platform emphasizes end-to-end automation, including damage detection, cost estimation, and workflow integration with insurer systems. As shown in Figure 6, Tractable also highlights adaptive AI systems capable of improving assessment accuracy over time using real-world repair and insurance data. However, the platform operates primarily as a B2B enterprise solution rather than a consumer-facing application.

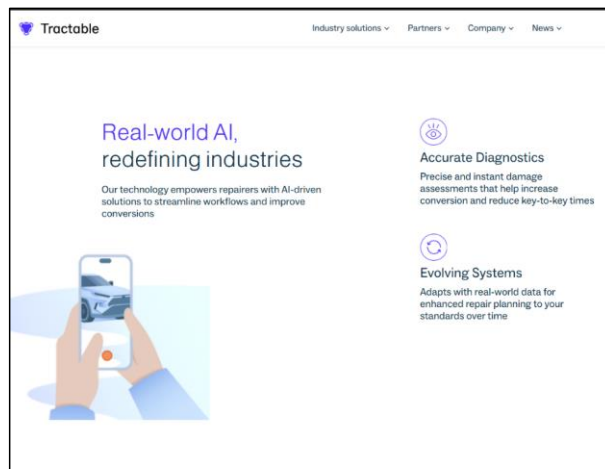


Figure 6: AI interface showing AI-based vehicle damage assessment [28].

Inspektlabs



Inspektlabs [29] is an AI-based vehicle inspection platform that provides automated damage assessment and vehicle condition analysis solutions for insurance, vehicle rental and leasing, and vehicle remarketing operations.

As illustrated in Figure 7, the platform uses a workflow-based process where 360-degree vehicle video capture is integrated with the Inspektlabs API to generate automated outputs such as damage assessment, objection evaluation, vehicle valuation, odometer reading, and fuel meter analysis.

The platform applies computer vision and AI-driven image analysis to perform automated inspections through smartphone-based or fixed-camera image capture systems. It supports several insurance-related functions, including objections assessment, fraud detection, severity estimation, vehicle triage, and repair cost estimation, while also providing OCR capabilities such as VIN and license plate recognition. Additionally, the system supports condition reporting, damage tracking, and automated workflow management for fleet, leasing, and remarketing operations.

Overall, Inspektlabs focuses on scalable AI-powered vehicle inspection and reporting workflows across multiple automotive sectors rather than targeting individual consumers directly.

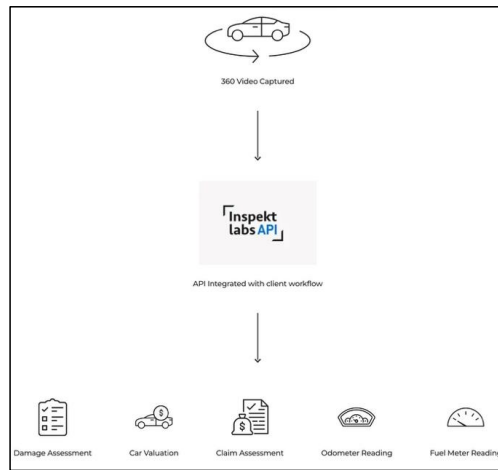


Figure 7: Inspektlabs Workflow [29].

Ravin AI



Ravin AI [30] is an AI-based vehicle inspection platform that provides automated damage assessment and vehicle condition analysis solutions for insurance providers, automotive remarketing businesses, and vehicle service centers.. As illustrated in Figure 8, the platform uses a workflow-based inspection process that combines vehicle image capture, automated video analysis, AI-based damage detection, and advanced imaging technologies to generate detailed vehicle condition reports.

The platform applies computer vision and machine learning techniques to detect vehicle damage, assess vehicle condition, and automate inspection reporting. For insurance providers, Ravin AI supports objections inspection, damage assessment, repair decision support, and cost estimation through the Ravin Inspect™ application. In fleet management and automotive remarketing, the platform provides automated inspections, condition grading, 360-degree imaging, and condition monitoring using the Ravin AutoScan™ system and mobile inspection tools. Ravin AI also offers

automated vehicle scanning solutions for automotive service centers, enabling detailed inspection reporting and improved customer communication.

Overall, Ravin AI focuses on scalable AI-powered vehicle inspection and reporting workflows across multiple automotive sectors rather than solely targeting insurance objection settlement processes.

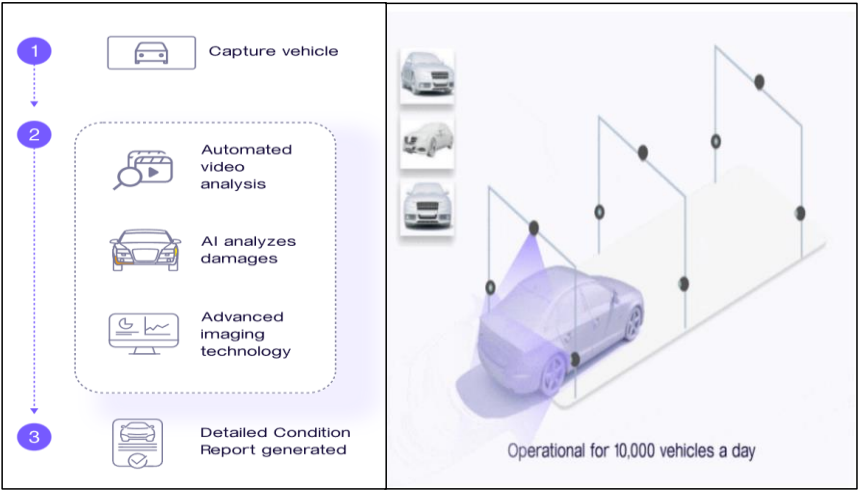


Figure 8: RavinAI Workflow [29].

Taqdeer



Taqdeer [8] is a Saudi-based vehicle damage assessment service operating under regulated valuation frameworks. Unlike AI-driven global platforms, Taqdeer relies primarily on certified human assessors who conduct physical inspections at authorized centers. As illustrated in Figure 9, the service follows a structured workflow in which vehicle owners book an appointment at a Taqdeer center, undergo vehicle damage assessment, and receive an official electronic report used for insurance and repair procedures.

The system produces standardized electronic damage assessment reports used by insurance companies to determine compensation and repair scope. While Taqdeer utilizes centralized electronic reporting systems for documentation and record management, damage detection and cost estimation are performed manually by accredited professionals rather than automated AI models.

As a locally regulated system, Taqdeer is fully integrated into the Saudi insurance ecosystem and serves individual vehicle owners directly. Its operational model emphasizes regulatory compliance, standardized valuation procedures, and formal assessment workflows within the national insurance framework.



Figure 9: Taqdeer Workflow [8].

Comparison

The comparison table highlights the key differentiators between CrashLens and existing solutions in the vehicle damage assessment market. As shown, Taqdeer lacks most automated capabilities, relying heavily on manual processes, which makes it limited in efficiency and scalability. In contrast, Tractable AI, Inspektlabs, and RavinAI offer comprehensive feature sets; however, these platforms are primarily designed for enterprise-level clients such as large insurance companies, making them less accessible to individual users. CrashLens matches these competitors across all five features as shown in Table 6 while specifically targeting individual users within the Saudi market through a mobile-first, localized solution. This positions CrashLens as the only platform that combines full automation capabilities with accessibility for everyday users, including an OCR feature that automatically extracts vehicle information from the registration card, addressing a gap that enterprise-focused competitors leave unserved in the Saudi context.

CrashLens focuses on improving accessibility to vehicle damage assessment by bridging the gap between enterprise-oriented global solutions and traditional manual assessment services.

Table 6: Feature comparisons

Feature	CrashLens	Taqdeer	Tractable AI	Inspektlabs	RavinAI
Target users	Individuals	Individuals	Enterprise	Enterprise	Enterprise
Automated Damage Detection	✓	✗	✓	✓	✓
Damage localization (bounding boxes / segmentation)	✓	✗	✓	✓	✓
Automated Severity Classification	✓	✗	✓	✓	✓
Automated Repair Cost Estimation	✓	✗	✓	✓	✓
Structured Digital Report Generation	✓	✓	✓	✓	✓
Real-Time Image Capture Guidance	✓	✗	✓	✓	✓
OCR-Based Vehicle Data Extraction	✓	✗	✓	✓	✓
User Objection Submission System	✓	✓	✗	✗	✗
Administrative Case Review & Approval	✓	✓	✓	✓	✓
Saudi Market Localization	✓	✓	✗	✗	✗
Case Status Tracking	✓	✓	✓	✓	✓
Report Download (PDF / Digital Export)	✓	✓	✓	✓	✓

4 System Description

This section describes the overall structure and functionality of the CrashLens system. It introduces the main system components, defines user roles, and explains how users interact with the application within the vehicle damage assessment and verification process.

4.1 Users

CrashLens includes two main user types: User and Admin. Each user type has its functionalities based on its role within the system.

4.1.1 Individual Users (Vehicle Owners)

Individual users are vehicle owners aged 18 years and above who have experienced a car accident and require a damage assessment and repair cost estimation. They use the mobile application to capture images of the damaged parts of the vehicle and enter the accident reference number.

Users are expected to have:

- Basic smartphone usage skills
- A general educational background (no specialized knowledge required)
- Arabic proficiency, since the application interface is in Arabic

Their knowledge of insurance procedures may vary; therefore, the system is designed to simplify the assessment process and reduce reliance on manual inspections.

4.1.2 Administrators (Admin)

Administrators are responsible for supervising and managing the overall system through a dedicated dashboard. Their role includes reviewing reports, monitoring system performance, and handling user objections.

They are expected to have:

- Experience in managing digital platforms or administrative systems
- Background of vehicle damage types and assessment procedures
- Moderate technical literacy

- Arabic proficiency to operate the system effectively

4.2 Requirements Elicitation and Analysis

The requirements for the proposed vehicle damage assessment system were gathered through interviews with domain experts (see [Appendix A](#)). Interviews were conducted with professional vehicle damage assessors working in certified assessment centers in different branches of taqdeer workshops. This approach ensured the collection of reliable, experience-based insights that accurately reflect real-world assessment workflows and challenges.

The elicitation process focused on identifying system needs, operational constraints, performance expectations, and potential improvement opportunities to support the development of an efficient and intelligent damage assessment platform. A total of three professional vehicle damage assessors participated in the interviews. All participants were actively employed in certified assessment centers at the time of the study and possessed practical experience in vehicle damage evaluation, including image documentation, spare part pricing coordination, labor cost estimation, and objection handling.

The interviews were conducted in a semi-structured format, allowing participants to elaborate on their daily workflows and professional challenges. The discussions covered several key areas, including the current damage assessment procedure, handling hidden and internal damage, repair cost calculation methods, factors affecting cost estimation, spare part pricing mechanisms, common operational challenges, assessment time requirements, difficult damage types, suggested system improvements, and the experience of electronic remote assessment during peak seasons such as Hajj.

The findings highlighted several important operational realities. First, the current assessment process depends heavily on structured photographic documentation, requiring images from multiple angles such as close-up damage images, dashboard images, odometer readings, seatbelt images, and chassis numbers. Second, only visible damage is evaluated during the initial inspection, while hidden or internal damage identified later may be submitted through an objection.

The interviews also revealed that repair cost estimation depends on multiple factors, including spare part type, paint type, vehicle age, labor costs, and previous compensation history. In addition, several challenges were identified, particularly in detecting mechanical and electrical

damage, distinguishing pre-existing damage from accident-related damage, and managing high case volumes during peak periods.

These findings directly influenced the functional and non-functional requirements of CrashLens, particularly the inclusion of guided image capture, AI-based damage detection, objection submission workflows, and administrative case review functionalities

4.2.1 User Interactions

The user interactions with the CrashLens system are represented using a Use Case diagram, as shown in Figure 10.

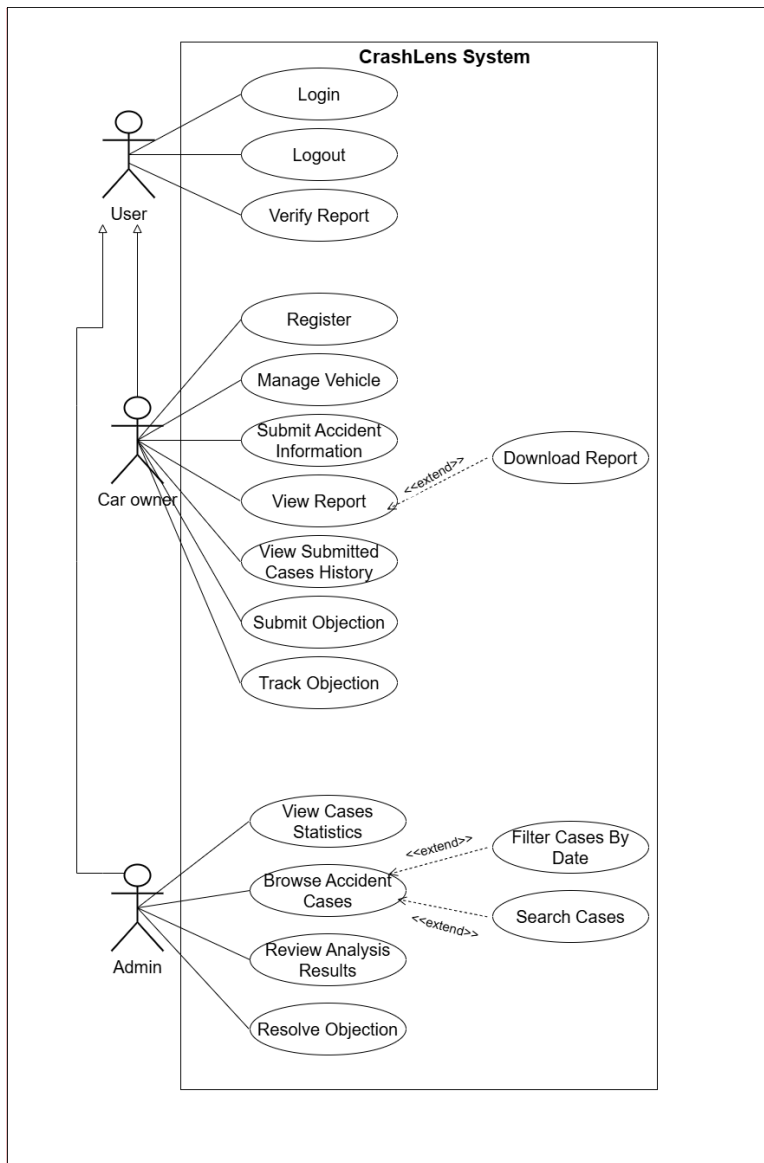


Figure 10: Use case diagram of Crashlens

Commented [46]: correct review in review analysis results.
Capitalize the first letter all usecases

4.3 Product Backlog

The following Table 7 displays the Definition of Ready table for the CrashLens system.

Table 7: Crashlens Definition of Ready

CrashLens Definition of Ready	
✓	Details are sufficiently understood
✓	Required dataset and image inputs are available and validated
✓	Dependencies are identified; no blocking dependencies exist
✓	Estimated and small enough to be completed during sprint
✓	Acceptance criteria are clear and testable
✓	Performance criteria are defined and testable
✓	Team understands how to demo the completed PBI

4.4 Product Backlog Table

The following Table 8 displays the Product Backlog table for the CrashLens system, including the user stories for both functional and non-functional requirements.

Table 8: Crashlens Product Backlog Table

PBI #	PBI	Size	Type	status	Acceptance Criteria
1	As a new user, I want to register in CrashLens so that I can access the CrashLens system.	3	Feature	Done	If the user enters a valid phone number, National ID and name then clicks “Continue”, then the system should send an OTP to that number. If the user enters the correct OTP within the allowed time, then the system should check

Commented [7]: We don't know what this means

					whether the phone number already exists. If the phone number does not exist, then the system should create a new user account and log the user in automatically. If the account is created successfully, then the user should be redirected to the home page. If the phone number or national ID is invalid or empty, then the system should display a validation message and prevent sending the OTP. If the OTP is incorrect or expired, then the system should display an error message and allow the user to request a new OTP.
2	As a registered user, I want to log into my account so that I can manage my accident cases.	3	Feature	Done	If the user enters a valid phone number and clicks “Continue”, then the system should send an OTP to that number. If the user enters the correct OTP within the allowed time, then the system should check whether the phone number already exists. If the phone number exists, then the system should authenticate the user and log the user in automatically. If the phone number doesn't exist display a message indicating the user to sign up.

					If login succeeds, then the user should be redirected to the home page. If the phone number is invalid or empty, then the system should display a validation message and prevent sending the OTP. If the OTP is incorrect or expired, then the system should display an error message and allow the user to request a new OTP.
3	As user, I want to log out so that my account remains secure.	1	Feature	Done	If the user selects the “Log Out” option from the home page, then the system should end the current session and redirect the user to the login page.
4	As a user, I want to add my vehicle information so that I can register my car in the system and use it later when creating accident cases.	3	Feature	Done	If the user opens the “Add Vehicle” page, then the system should display fields for entering vehicle details (e.g., make, model, year, plate number, and color). If the user fills in all required fields and submits the form, then the vehicle should be saved successfully to the user’s account. If any required field is missing, then the system should display validation messages and prevent submission. If the vehicle is added successfully, then it should appear in the user’s vehicle list and be available for selection when creating accident cases.

5	As a user, I want to scan my vehicle registration card to automatically extract and save my vehicle information.	5	Feature	Done	If the user selects “Scan Vehicle Registration”, then the camera should open to capture the registration document. If the image is captured clearly, then the system should run OCR to extract vehicle information (e.g., plate number, vehicle type, model year, and color). If the extracted data is detected successfully, then the fields should be auto filled and shown to the user for review. If the OCR fails to detect some fields, then the user should be allowed to edit or manually complete the missing information. If the user confirms the extracted data, then the vehicle should be saved to the user’s account.
6	As a user, I want to delete a saved vehicle from my account so that I can remove vehicles I no longer use.	3	Feature	Done	If the user selects a vehicle from their vehicle list and chooses the delete option, then the system should display a confirmation message before deletion. If the user confirms the deletion, then the vehicle should no longer appear in the user’s vehicle list but should be archived rather than permanently deleted. If the vehicle is archived, then the system must retain its information, so it continues to appear in all related reports.

7	As a user, I want to submit my accident case so that it can be analyzed by the system.	3	Feature	Done	If the user clicks the “Create New Case” button, then the system should open a new accident case form. A valid Najm report must be uploaded in order to submit an accident case. If an invalid report is detected, the system displays an error message prompting the user to re-upload a correct report. If the user completes all required information and uploads at least one image and at most 10 images , then presses the Submit button, the system should send the case data and images to the server for analysis. If the submission is successful, then the user should see a confirmation message and the case status should change to “Under Analysis.”
8	As a user, I want to capture vehicle accident damage photos using my camera so that the system can analyze the accident.	5	Feature	Done	If the user selects “Capture Damage Photos”, then the system should open the camera to take images of the vehicle. If the photo is clear and captured successfully, then it should be uploaded and attached to the accident case. If the image quality is poor (e.g., blurry or too dark), then the system should display guidance instructions to help the user capture a clearer image before proceeding.

					If multiple damages are present, then the system should allow capturing several images to cover all damaged areas.
9	As a user, I want to remove an uploaded image before submission so that I can correct mistakes.	2	Feature	Done	If the user views the uploaded images before submission, then each image should display a “Remove” option. If the user selects the “Remove” option on an image, then the image should be deleted from the case preview. If the user removes all images, then the system should prevent submission until at least one image is uploaded.
10	As a user, I want guidance while capturing photos so that I can submit clear and valid images.	5	Feature	Done	If the user opens the camera to capture photos, then the system should display on-screen guidance. If the lighting is too low or too bright, then the system should show a warning asking the user to adjust lighting. The system shall provide real-time visual guidance instructions throughout the image capture process to assist the user in obtaining clear and valid photos.
11	As an ML engineer, I want to train a YOLO-based model so that the system can detect vehicle damage.	8	Technical Work	Done	Model is trained. Validation metrics are recorded. Model achieves acceptable mAP threshold.

12	As a user, I want the system to detect damaged areas in my submitted images so that I can visually understand the affected parts.	8	Feature	Done	If the user submits an accident case with vehicle images, then the system should analyze the images to detect damaged areas. If damaged areas are detected, then the system should highlight the damage visually (e.g., using bounding boxes). If multiple damaged areas are present, then the system should detect and display all identified regions.
13	As an ML engineer, I want to train a model to classify damage severity so that the system can estimate impact level.	8	Technical Work	To do – will be implemented in release 2	Severity categories (e.g., minor, moderate, severe) are defined. Mapping logic between detection output and severity is documented. Threshold values are specified. Model is trained and validated. Performance metrics are recorded. Model reaches acceptable accuracy threshold.
14	As a user, I want to see the severity level of my vehicle damage so that I understand the seriousness of the accident.	3	Feature	To do – will be implemented in release 2	If the analysis is completed, then the system should display the damage severity level (minor, moderate, or severe) in the case results.
15	As a user, I want to see	3	Feature	To do – will be	If the analysis is completed, then the system should display

	the estimated repair cost so that I can understand the expected financial impact.			implemented in release 2	the estimated repair cost range in the case results. If the case is still under analysis, then the cost estimate should not be shown and the status should indicate processing.
16	As a user, I want a structured report generated after analysis so that I can review my accident assessment clearly.	5	Feature	To do – will be implemented in release 2	If the analysis is completed for an accident case, then the system should automatically generate a structured report for that case. If the report is generated, then it should include the user's basic information, vehicle information, uploaded images, detected damaged areas, severity classification (minor, moderate, severe), and the estimated cost range. If the report includes visual results, then it should display the highlighted damaged regions (e.g., bounding boxes or masks) for clarity. If the user opens the report, then the report should be viewable inside the application. If the report is generated successfully, then it should be saved and linked to the accident case for future access.
17	As a user, I want to access my previous reports so that I can review past accident cases.	3	Feature	To do – will be implemented in release 2	If the user opens the Case History section, then the system should display a list of all previously submitted accident cases with their report status. If the user selects a specific case, then the system should open the associated report and

					display the saved assessment results. If a case is still under analysis, then the system should show the current status and indicate that the report is not yet available. If the user has no previous cases, then the system should display a clear message indicating that no reports are available.
18	As a user, I want to download my assessment report as a PDF so that I can share it if needed.	3	Feature	To do – will be implemented in release 2	If the user opens an assessment report, then the system should provide an option to download the report as a PDF file. If the user selects the Download PDF option, then the system should generate and download the report in PDF format to the device. If the report generation is successful, then the downloaded file should contain all report details and visual results.
19	As a third party, I want to verify the accident report through a QR code so that I can confirm its authenticity and accuracy.	5	Feature	To do – will be implemented in release 2	If the third party scans the QR code included in the report, then the system should open a secure verification page for that report. If the report exists and is valid, then the system should display the report details.
20	As an admin, I want to browse submitted accident	3	Feature	To do – will be implemented in release 2	If the admin opens the Cases Management section, then the system should display a list of all submitted accident cases.

	cases so that I can manage incoming assessments.				If cases exist, then each case should show basic information (case ID, user, date, and current status). If the admin selects a specific case, then the system should open the full case details and associated images. If there are no submitted cases, then the system should display a message indicating that no cases are available.
21	As an admin, I want to filter accident cases by date so that I can quickly locate cases within a specific time range.	3	Feature	To do – will be implemented in release 2	If the admin opens the Cases Management page, then the system should provide date filter fields. If the admin selects a date range, then the system should display only cases submitted within that period. If the admin clears the date filter, then the system should display all cases again.
22	As an admin, I want to search for a case so that I can directly access related cases.	3	Feature	To do – will be implemented in release 2	If the admin enters a valid vehicle plate number or case ID in the search field, then the system should display all cases associated with that plate number. If no cases exist for the entered vehicle plate number or case ID, then the system should show a message indicating that no matching cases were found. If the admin clears the search input, then the system should

					return to displaying the full list of cases.
23	As an admin, I want to view case statistics so that I can monitor system performance and case trends.	5	Feature	To do – will be implemented in release 2	If the admin opens the Statistics Dashboard, then the system should display summarized case statistics (e.g., total cases, approved cases, under analysis). If data exists, then the system should present visual charts showing trends over time (e.g., cases per day/month). If the dashboard loads successfully, then all statistics should reflect the latest system data.
24	As an admin, I want to review a case to ensure the assessment is accurate before finalization.	3	Feature	To do – will be implemented in release 2	If the admin opens the Cases page and selects a specific case, then the system should display the full case details and the generated report. If the admin reviews the report, then they should have the option to either modify the report or approve the case. If the admin chooses to modify the report, then the system should allow editing of the assessment details and save the updated report. If the admin approves the case, then the system should mark the case status as Approved. If the admin completes either action, then the system should update the case status in the user's account.

25	As a user, I want to submit an objection if I disagree with the result so that my case can be re-evaluated.	3	Feature	To do – will be implemented in release 2	If the user opens the objection page, then the system should display a list of eligible cases for objection. If the user selects a case, then the system should allow them to enter the objection reason. If the user submits the objection successfully, then the objection status should change to Under Review.
26	As a user, I want to track the status of my submitted objection so that I can stay informed about its review progress.	3	Feature	To do – will be implemented in release 2	If the user opens the objection section, then the system should display a list of their submitted objections with their current statuses. If an objection is under review, then the system should display the status as “Under Review”. If the objection is accepted, then the system should update the status to “Accepted”. If the objection is rejected, then the system should update the status to “Rejected”. If the admin updates the objection decision, then the status should be updated immediately in the user’s account.
27	As an admin, I want to review user objection so that I can decide whether	5	Feature	To do – will be implemented in release 2	If the admin opens the objection page, then the system should display a list of submitted user objections.

	reassessment is needed.				If the admin selects a specific objection, then the system should display the related case details and objection reason. If the admin accepts the objection , then the system should allow the admin to modify the report and update the case status to “Updated”. If the admin rejects the objection, then the system should keep the report unchanged and mark the objection as “Rejected”. If a decision is made, then the system should update the case status in the user’s account.
Non-Functional Requirement					
28	As a first-time user I want the app to be simple so that I would be able to complete the accident case process within 5 minutes under normal operating conditions.	3	Feature	In Progress – full implementation of the system not finished yet	If a first-time user follows the normal accident case submission workflow under standard network conditions, then the complete process shall not exceed 5 minutes from case creation to final submission. If required information or images are missing during submission, then the system shall immediately display clear validation messages within 5 seconds. The submission process should not require unnecessary steps
29	As a user, I want the system to be available at least 99% of the time so that I can access my accident	3	Feature	In Progress – full implementation of the system not finished yet	If a user attempts to access the system, then the system should be operational and accessible 99% of the time measured monthly. If scheduled maintenance occurs, then users should be notified in advance.

	reports whenever needed.				If downtime exceeds the allowed threshold, then the event should be logged and reported.
30	As a user, I want the application pages to load within 10 seconds from the moment I initiate a request to access any page, so that I can access all functions without delays.	3	Feature	In Progress – full implementation of the system not finished yet	If user open any page, then the page should fully load within 10seconds under normal network conditions on supported mobile devices.
31	As a user, I want the app interface to adapt to different screen sizes so that I can complete tasks without layout issues.	3	Feature	In Progress – full implementation of the system not finished yet	If the app is opened on different supported screen sizes, then the UI should scale and display all essential elements without overlapping, truncation, or unreadable text. If a screen becomes too small for a specific layout element, then the system should apply responsive behaviour (e.g., stacking components vertically) rather than hiding required actions.
32	As a user, I want the application to show helpful error messages and handle unexpected errors smoothly so that I can easily understand and resolve issues.	2	Feature	In Progress – full implementation of the system not finished yet	If the user performs an action with missing or invalid input, then the system should display a clear and understandable error message explaining the issue.

5 System Design

5.1 Architectural Diagram

The CrashLens system follows the Model–View–Controller (MVC) architectural pattern to ensure clear separation between the user interface, application logic, and data processing components. This architecture was selected because it improves maintainability, supports code reusability, and simplifies system development and management. As illustrated in Figure 11, the MVC architecture separates the system into three main components:

1. **Model:** It is the part that deals with data and processing it. This part can be used across multiple views.
2. **View:** This part is mainly associated with the User Interface (UI), and it is used to provide the visual representation of the model's data.
3. **Controller:** It acts as an intermediary between the model and the view. This part can be used with different views.

The user interacts with the system through the view component by uploading vehicle images and submitting vehicle-related information. The controller receives these requests from the view and forwards them to the model component for processing. Within the model, the backend API and AI engine analyze the submitted images, generate damage assessment results, and store the processed data in the database. When data needs to be displayed, the controller retrieves the required information from the model and forwards it to the view for presentation to the user or administrator

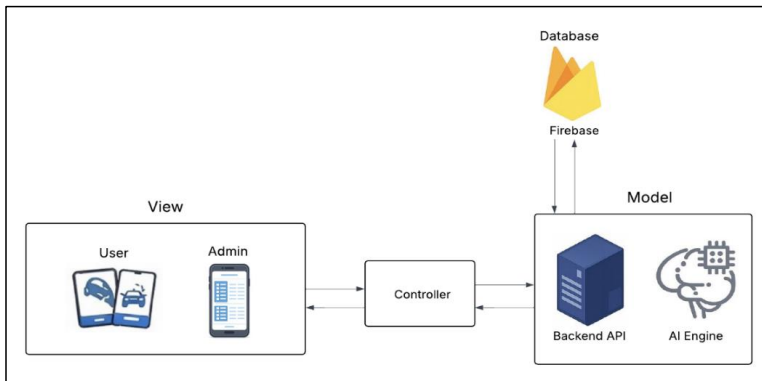


Figure 11: Crashlens MVC architecture

5.2 Class Diagram

The class diagram represents the structural design of the CrashLens system by illustrating the main classes, their operations, attributes, and interactions. Figure 12 presents the conceptual class diagram for CrashLens.

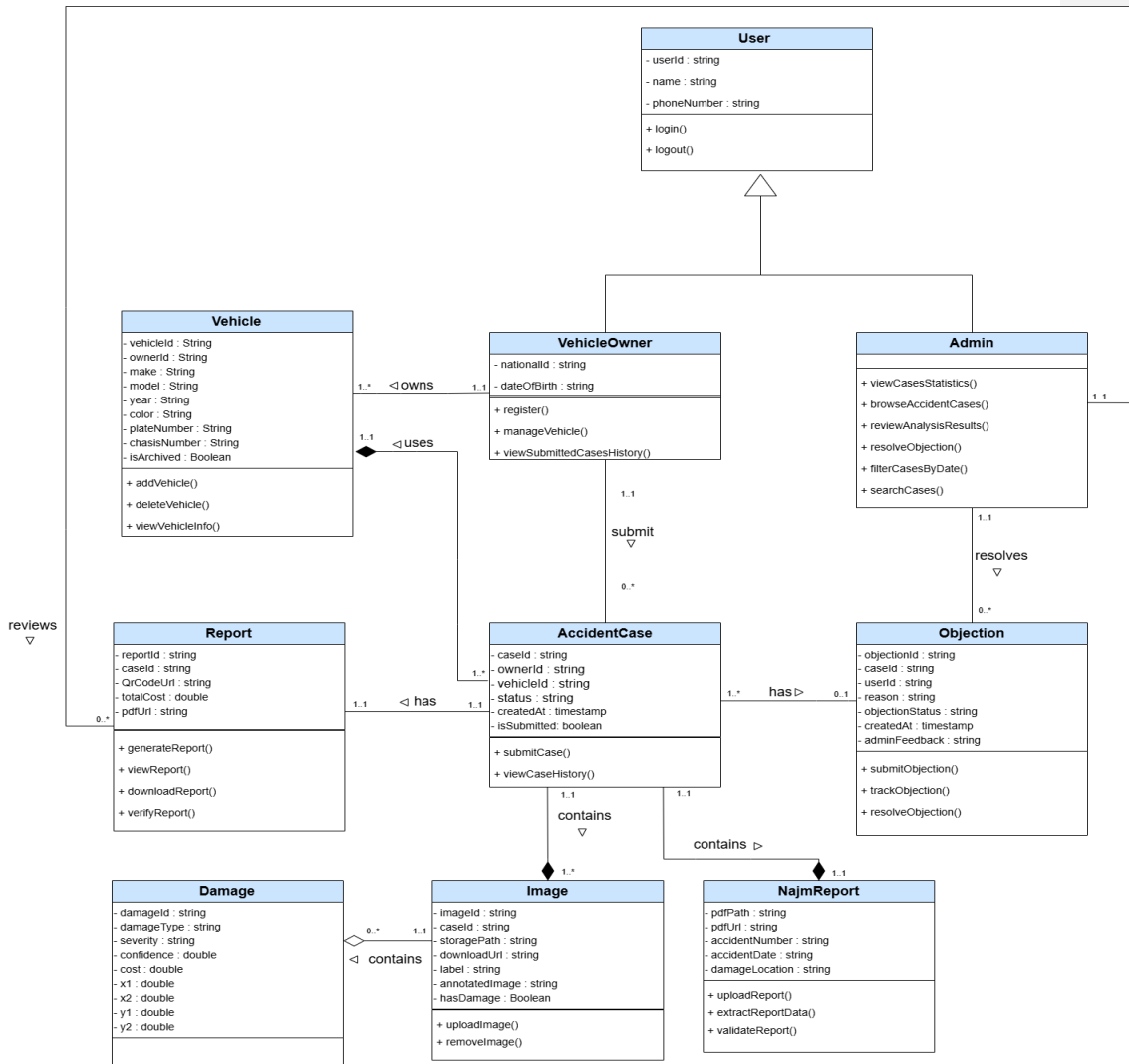


Figure 12: Class Diagram of CrashLens

5.3 Component Level Design

General pipeline for Crash Lens

Figure 13 illustrates the high-level workflow of the AI-based vehicle damage assessment pipeline used in CrashLens. The process begins with users submitting vehicle damage images through the application. The system then analyzes the images to detect damaged regions and assess the severity of the detected damage. Based on the assessment results and vehicle-related information, an estimated repair cost is generated. Finally, the generated assessment results and reports are stored in the database and presented to the user through the application interface.

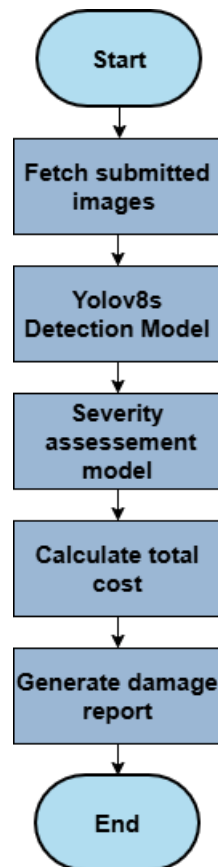


Figure 13: General Pipeline of Crashlens

Commented [48]: too small, many repeated steps.

Commented [49R8]: make it high level

Commented [410R8]: main steps only

Adding Vehicle

Figure 14 displays the flowchart of vehicle addition functionality using either OCR-based extraction or manual data entry. Users can choose between scanning the vehicle registration card through OCR processing or entering the vehicle information manually. The system validates the extracted or entered data before storing the vehicle record in the database and linking it to the current user account.

Commented [411]: redraw better, font size is very small!!!

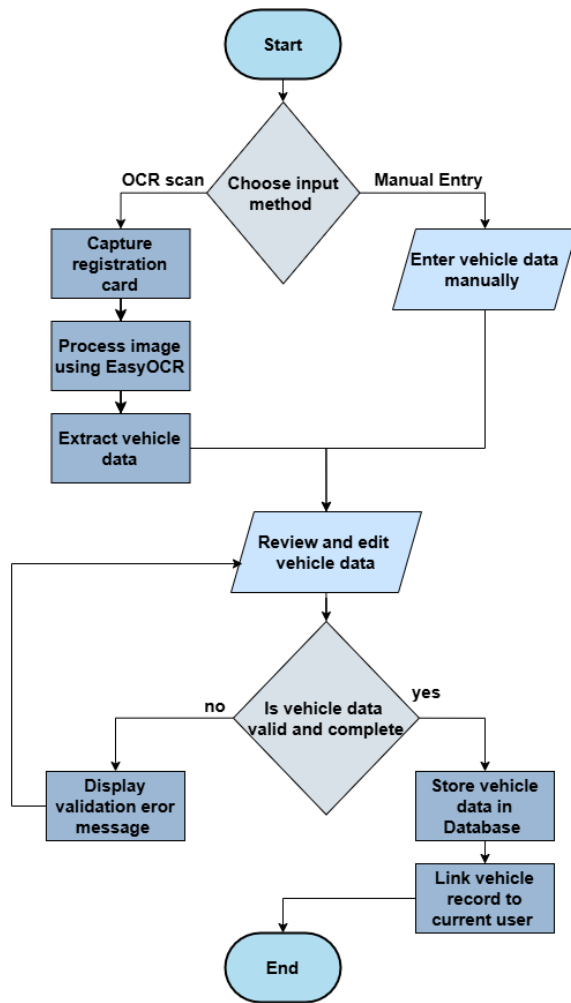


Figure 14: Vehicle Addition Flowchart

Submit accident case

Figure 15 displays the flowchart of the accident case submission functionality. The process begins with uploading the Najm accident report and performing OCR-based extraction and verification of the required information. Once the report is verified successfully, the user can upload vehicle damage images and submit the accident case for further assessment processing.

Commented [412]: font too small.
Review the steps carefully:
run OCR
Extract the information
perform report verification
the condition is Verified?

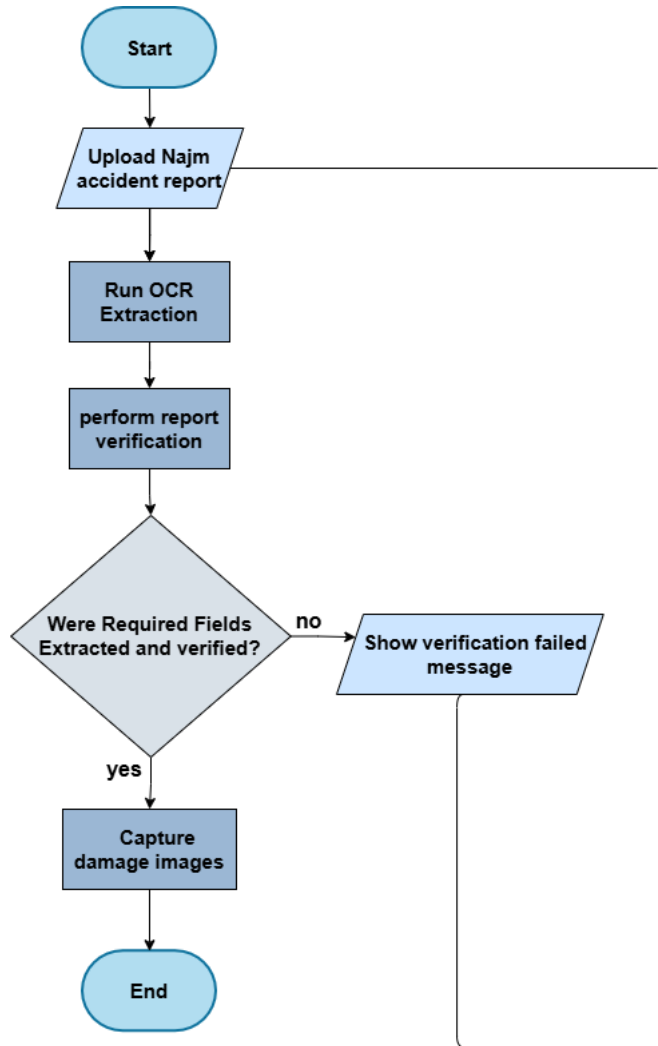


Figure 15: Submit Accident Case Flowchart

5.4 Data Design

5.4.1 Data Models

ER Diagram

Figure 16 displays the Entity Relationship Diagram (ER) of the CrashLens system.

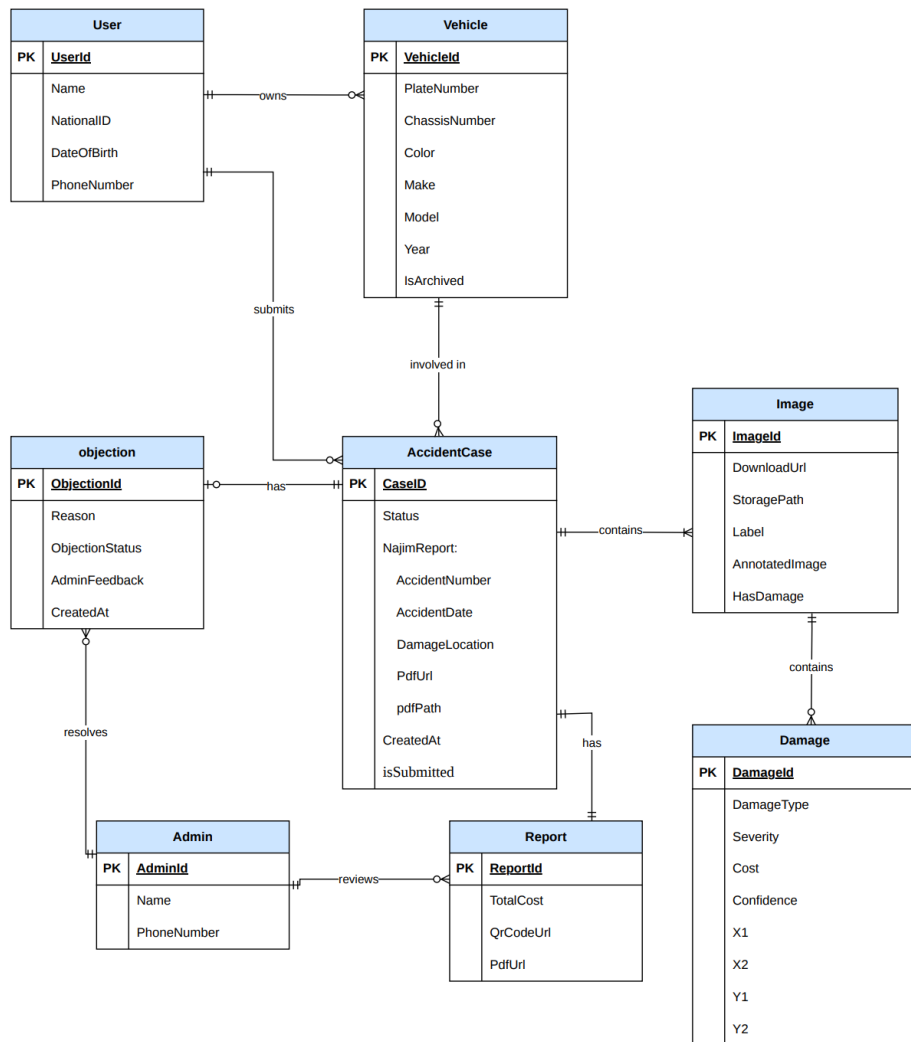
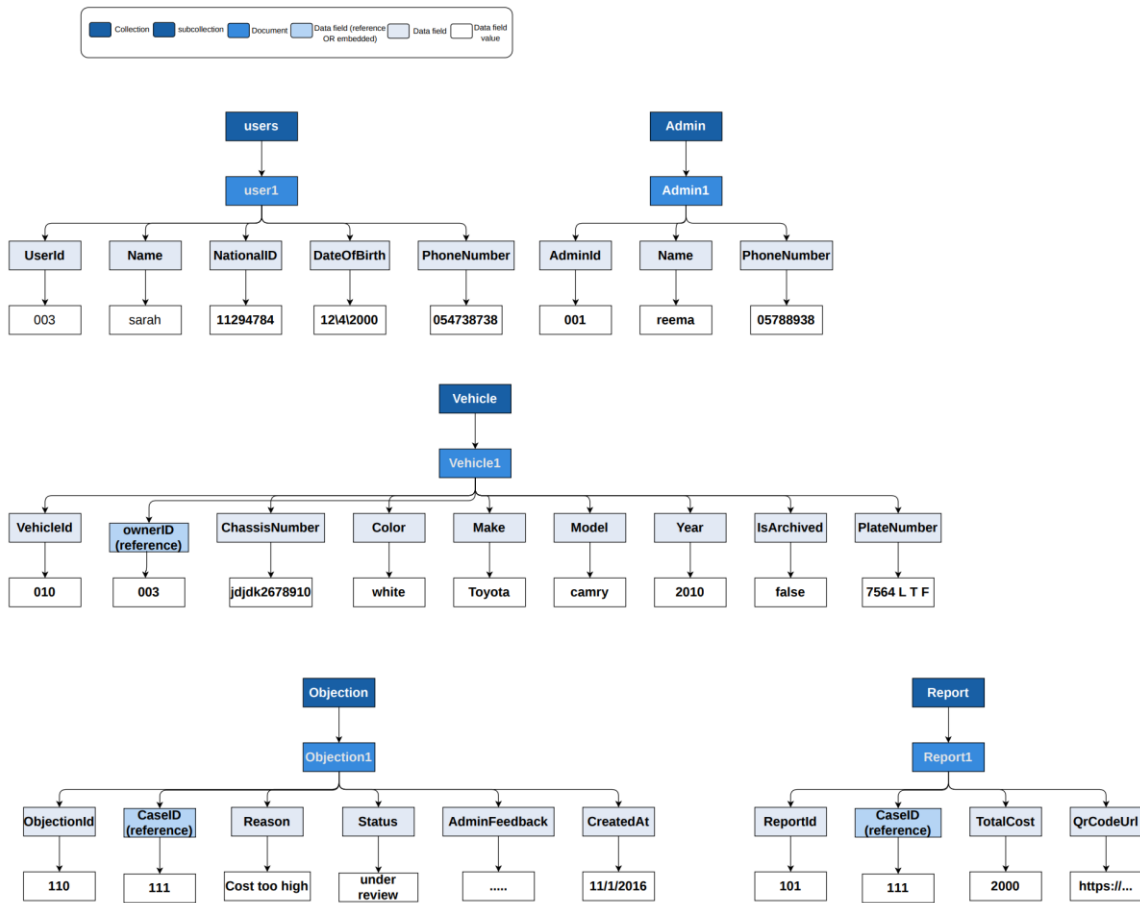


Figure 16: ER diagram.

None Relational Data Model

Figure 17 presents the non-relational data model of the CrashLens system. The model illustrates the hierarchical structure of collections, documents, and data fields, as well as the referencing relationships between related entities.



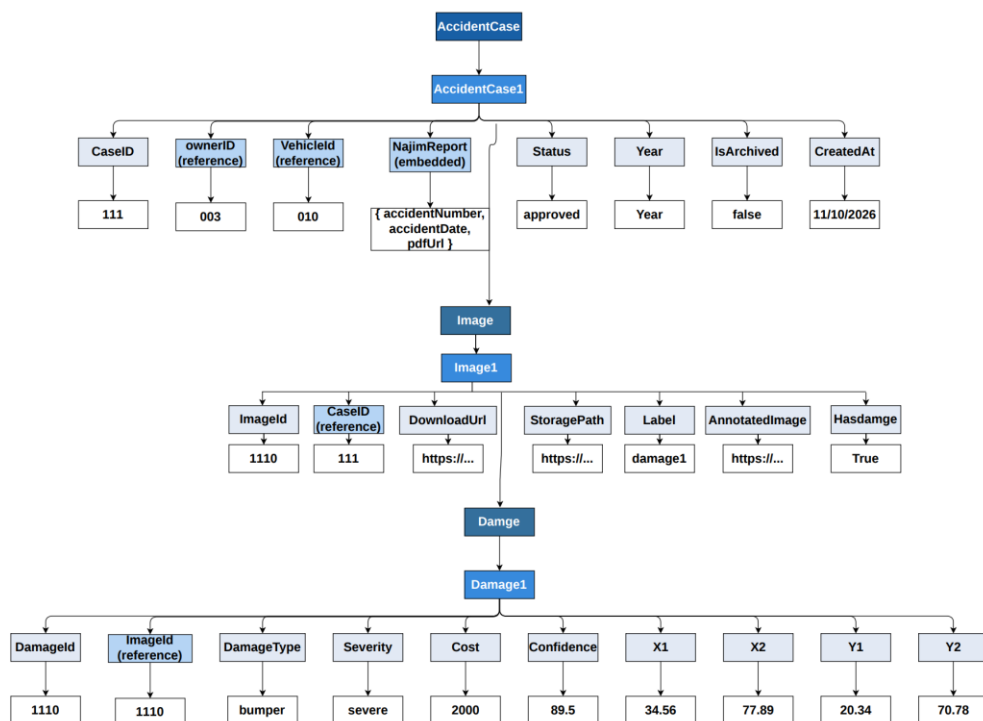


Figure 17: Non-Relational Data Model.

Commented [413]: the trees are not consistent in size, make it better for readability

5.4.2 Data Collection and Preparation

CrashLens utilized the CarDD dataset, which was pre-partitioned into three fixed splits: a training set, a validation set, and a test set. Sample vehicle damage images from the dataset are illustrated in Figure 18. Prior to feeding the data into the model, several preprocessing steps were applied. The annotations, illustrated in Figure 19, were first converted from their original COCO JSON format into YOLO-compatible text files, where each bounding box was transformed into a center-point format normalized by image dimensions and category IDs were remapped from COCO's one-based indexing to YOLO's zero-based scheme. All images were then resized to 640×640 pixels and normalized to the $[0, 1]$ range. Since baseline experiments revealed that three damage classes (dent, scratch, and crack) produced substantially lower detection accuracy due to

their subtle, low-contrast visual appearance, a targeted augmentation strategy was applied exclusively to training images containing at least one annotation from these classes. Only photometric operations were employed, including brightness and contrast adjustment, mild sharpening, and Gaussian noise injection, so that the original label files could be reused without modification. Each eligible training image received one additional augmented copy to boost representation of the difficult classes without over-inflating the training set. Finally, a verification step confirmed zero overlap between all three splits, ensuring no data leakage into evaluation.



Figure 18: CarDD Dataset Samples.

```

"categories": [
  {
    "id": 1,
    "name": "dent"
  },
  {
    "id": 2,
    "name": "scratch"
  },
  {
    "id": 3,
    "name": "crack"
  },
  {
    "id": 4,
    "name": "glass shatter"
  },
  {
    "id": 5,
    "name": "lamp broken"
  },
  {
    "id": 6,
    "name": "tire flat"
  }
],
"images": [
  {
    "id": 1,
    "width": 1000,
    "height": 750,
    "file_name": "000001.jpg",
    "license": 0
  },

```

Figure 19: Annotation Sample

Interface Design

Site Map

Figure 20 displays the sitemap of the CrashLens mobile application

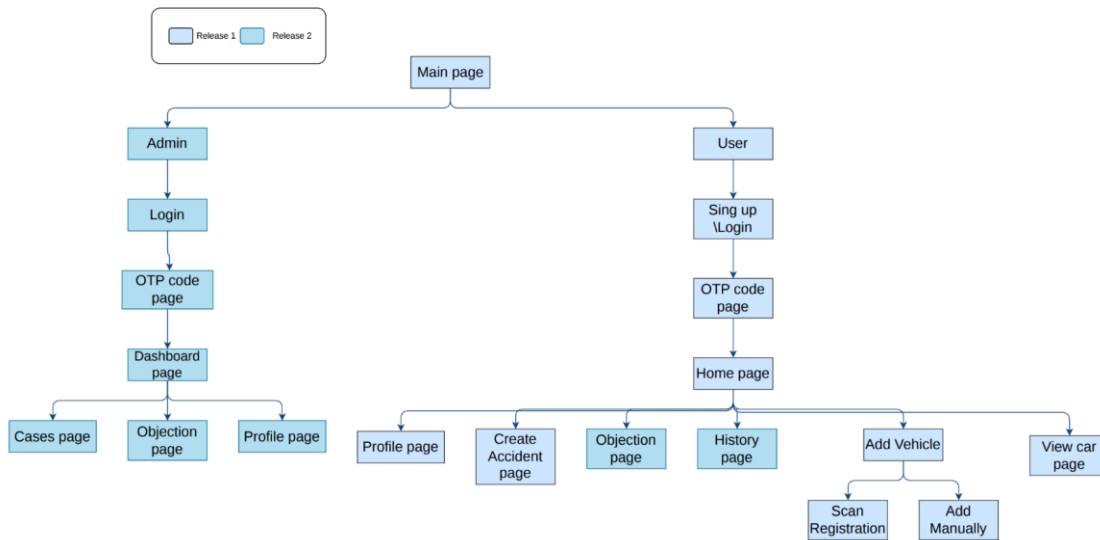


Figure 20: Mobile Application Site Map.

UX guidelines

In CrashLens, the user experience is designed to support users through a multi-step process that includes report verification, vehicle registration, and damage analysis. To ensure ease of use and reliability, the application follows the following UX principles:

1. Familiarity

The system uses commonly recognized icons, standard navigation patterns, and familiar terminologies similar to those used in existing applications. This design approach helps users quickly understand the interface and interact with the system with minimal learning effort. Consistent wording and intuitive visual elements were also used throughout the application to improve usability and provide a more predictable user experience. As illustrated in Figure 21, the interface demonstrates these familiarity principles through its design.



Figure 21:Familiarity interface

2. Consistency

Consistency is maintained across all screens to provide a smooth and predictable user experience. For example, the bottom navigation bar remains uniform throughout the application, allowing users to easily return to the home screen or access key features. Buttons, cards, and input fields follow the same styling and behavior across pages such as vehicle addition pages, case details page, and profile screen. A unified color scheme is also used to visually distinguish actions and system states (e.g., primary actions, alerts, and success indicators), helping users quickly interpret information. As illustrated in Figure 22, the interface demonstrates these consistency principles through its design



Figure 22: Consistency interface

3. User Guidance

The system actively guides users during critical steps, especially during image capture and registration card scanning. For instance, the guided camera interface provides real-time feedback on image quality (such as brightness and sharpness), ensuring that users capture suitable images for damage analysis. Similarly, clear instructions are displayed during registration card scanning. This guidance reduces user uncertainty and increases the accuracy of the collected data. As illustrated in Figure 23, the interface demonstrates these user guidance principles through its design.

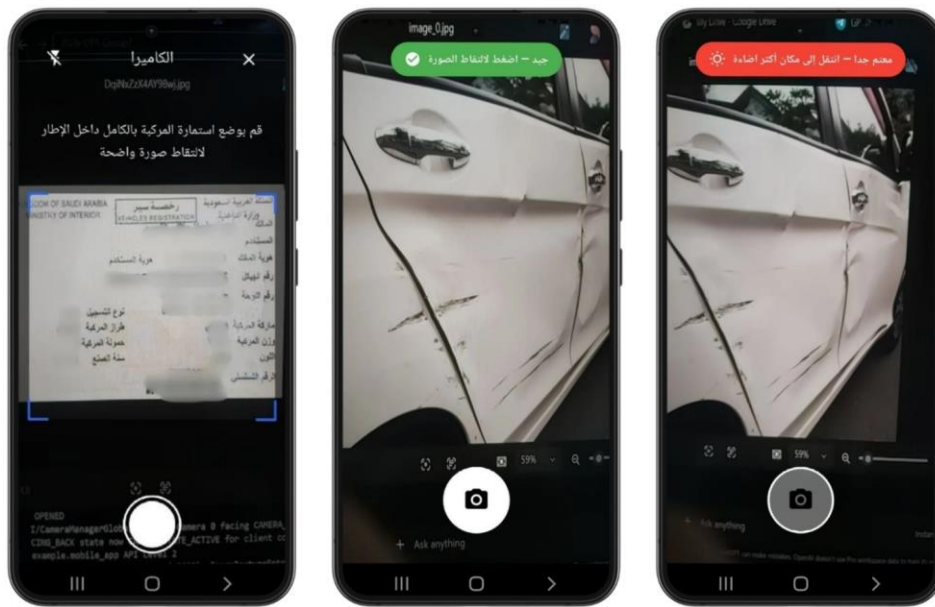


Figure 23: User Guidance interface

4. Feedback

The application provides immediate and clear feedback in response to user actions. Progress indicators are displayed during operations that require processing time, such as Najm report verification ,Damage Image upload ,Damage analysis, registration card scanning This ensures that users are always informed about the system status, reducing confusion and preventing repeated actions. As illustrated in Figure 24, the interface demonstrates these feedback principles through its design.

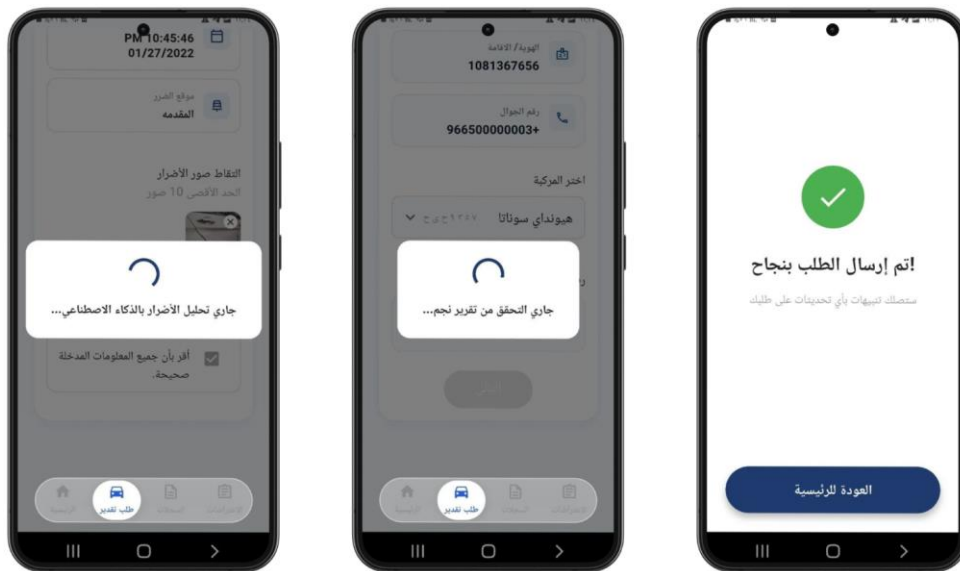


Figure 24: Feedback interface.

5. Error Prevention

To minimize user errors, the system implements validation checks before allowing submission of critical data. For example, required fields for registration must be completed before proceeding, and clear error messages are shown if any information is missing or incorrect. The same goes for possible case submission issues such as connection failure in which This approach ensures that the data sent for processing is complete and valid, improving both user experience and system reliability. As illustrated in Figure 25, the interface demonstrates these error prevention principles through its design



Figure 25: Error prevention interface.

6 System Implementation

6.1 Overview

CrashLens is a mobile application designed to assist users in submitting accident cases and estimating vehicle repair costs. The system allows users to register, manage their vehicles, upload accident reports, and capture images of vehicle damage.

The system follows the Model–View–Controller (MVC) architecture, where:

- the View is represented by the Flutter user interface;
- the Controller manages user interactions and application logic;
- the Model represents the data stored in Firebase services.

The application integrates backend services for processing uploaded documents using Optical Character Recognition (OCR). OCR is utilized in two main functionalities which are Najm report analysis, where key accident details such as accident date , number , and damage location are extracted and information like the user ID and vehicle plate number for validation , one more functionality using OCR is the Registration card scanning, where vehicle information is extracted automatically to assist users during vehicle registration that includes 6 important fields like the VIN , the vehicle plate number and the make and model of the vehicle as well as the color and finally the manufacturing year.

The system is designed to provide a responsive user experience by performing heavy processing tasks through backend services while maintaining smooth interaction on the mobile application.

6.2 Development tools and technologies

The mobile application was developed using Flutter with the Dart programming language. Firebase Authentication was used for user authentication and session management, Cloud Firestore for database storage, and Firebase Storage for storing uploaded reports and vehicle images.

The backend was implemented using FastAPI with Python and is responsible for OCR processing, data validation, and communication with Firebase services. Communication between the mobile application and backend services is performed through HTTP APIs and POST requests.

The AI-based damage detection component was developed using PyTorch and the Ultralytics YOLOv8 framework, while OpenCV and PIL were used for image preprocessing and manipulation. Model training and experimentation were conducted using Google Colab.

6.3 Challenges and Solutions

Early Verification of Najm Reports

A key challenge was ensuring that users upload valid Najm accident reports while minimizing delays in the submission process. Initially, verification was performed after full case submission, which led to poor user experience when errors were discovered late. To improve usability, the system flow was redesigned so that users first select a vehicle and upload Najm reports then proceed to the next step upon clicking “Next”, in which the frontend immediately calls the backend to perform OCR extraction of accident number, date and damage location, in addition to matching of national ID and plate number with the user’s stored data for verification purposes. If verification fails, the user is notified immediately and if verification is successful, the user proceeds to capture damage images. This approach ensures early validation and prevents invalid case submissions.

OCR Challenges in Registration Card Extraction

Extracting structured information from registration cards presented significant challenges due to Arabic text complexity, variation in layout and formatting and incorrect mapping of extracted values to fields. To address this, a refined extraction approach was implemented using Anchor-based extraction where specific keywords were used to locate regions of interest before extracting values, also Region-based preprocessing was used to extract the manufacturing year due to its extremely small size by cropping specific areas of the image before further zooming in and performing extraction.

Image Quality Assessment Challenge in User Guidance

Providing accurate real-time guidance for image capture presented a challenge, particularly in assessing image sharpness (blurriness). An initial approach was based on edge variation assumptions, where images with high edge intensity were considered sharp, while smoother images were classified as blurry. However, this method proved unreliable in real-world scenarios, as edge-based techniques are highly sensitive to noise, lighting conditions, and background complexity. Consequently, the system occasionally failed to provide consistent or

meaningful feedback to users. To address this limitation, a frequency-domain approach was adopted using Fast Fourier Transform (FFT). Instead of relying on spatial edge information, this method analyses the distribution of high-frequency components within the image. Sharp images typically contain more high-frequency details, while blurry images exhibit a loss of such components. By computing the ratio of high-frequency energy to total energy, the system achieved a more robust and reliable measure of image sharpness. This improvement significantly enhanced the accuracy and consistency of the guidance feedback provided to users during image capture.

Data Consistency and Validation

Maintaining consistency between user data, vehicle records, and accident reports was essential. The system enforces validation by matching user identity with Najm report data and verifying plate number and ensuring required fields are successfully extracted. Fallback mechanisms are also implemented in the UI show early flash messages to users to handle missing or inconsistent data gracefully.

6.4 Vehicle Damage Detection

The damage detection model was developed using the YOLOv8 architecture and trained on the preprocessed CarDD dataset to detect and localize six vehicle damage categories. We tested four different configurations as follow:

Commented [سا14]: مافهمت

Configuration 1: Baseline Model

The initial experiment used the lightweight YOLOv8n (Nano) architecture as a baseline configuration due to its low computational complexity and fast training speed. The model was trained for 100 epochs using an image size of 640×640 pixels, a batch size of 16, and an early stopping patience value of 20 epochs. The default SGD optimizer and standard YOLO training augmentations were used. The baseline model achieved acceptable detection performance on visually distinct damage categories such as glass shatter and tire flat. However, detection accuracy for scratches, dents, and cracks remained low due to their fine-grained appearance, low contrast, and irregular texture patterns.

Configuration 2: Resolution, Epoch, and General Augmentation Experiments

In the second configuration, several general training adjustments were tested to improve the baseline YOLOv8n model. First, different input image sizes were evaluated, including 640, 960, 1024, and 1280 pixels, while keeping the model architecture, batch size, and epoch count fixed. The purpose of this step was to determine whether higher image resolution could improve the detection of small and visually subtle damage regions. Based on the validation results, the 640×640 image size was retained for the following experiments.

After selecting the image size, the training duration was increased from 100 to 150 epochs, with the batch size kept at 16 and early stopping patience increased to 25. A separate experiment also applied general training-time augmentation, including rotation, translation, scaling, shearing, horizontal flipping, HSV color adjustments, and mosaic augmentation, as well as scaling augmentation specifically aimed to improve the model's robustness to vehicle damages appearing at different object sizes and viewing distances within the image. These augmentations were applied dynamically during training, meaning that no additional image files were permanently added to the dataset at this stage.

Although these changes slightly improved the overall validation performance, the hard classes (scratch, dent, and crack) continued to show lower detection performance compared with more visually distinct classes such as glass shatter and tire flat. This indicated that general augmentation and longer training were not sufficient to address the difficulty of detecting subtle damage patterns.

Configuration 3: Targeted Augmentation and Optimizer Adjustment

Since the previous experiments showed that the scratch, dent, and crack classes consistently underperformed, a targeted augmentation strategy was introduced specifically for these difficult categories. Unlike the general online augmentation used in the previous configuration, this phase employed an offline dataset expansion approach in which additional augmented image copies were physically generated and added to the training dataset.

Only training images containing at least one annotation belonging to the scratch, dent, or crack classes were selected for augmentation. A total of 2184 training images met this condition, and each image received one additional augmented copy, resulting in 2184 newly generated augmented images added to the training set. Because the applied transformations were photometric

rather than geometric, the original YOLO annotation files could be reused without requiring bounding box recalculation.

The augmentation pipeline included brightness adjustment, contrast enhancement, mild sharpening, and Gaussian noise injection. These operations were specifically chosen to improve the model's ability to learn subtle texture variations and low-contrast damage patterns commonly associated with fine-grained vehicle damage.

In addition to dataset augmentation, the optimizer was changed from SGD to AdamW to improve optimization stability and convergence behaviour for sparse and fine-grained visual features. The model was trained using the YOLOv8n architecture with an image size of 640×640 pixels, batch size of 16, learning rate of 0.001, weight decay of 0.01, and early stopping patience of 30 epochs across a total of 150 training epochs.

The targeted augmentation strategy improved feature representation for the difficult classes and produced more stable confidence scores compared with the earlier configurations, although overall performance limitations of the YOLOv8n architecture remained noticeable for fine-detail damage localization tasks.

Configuration 4: Final Model Architecture

In the final configuration, the lightweight YOLOv8n architecture was replaced with YOLOv8s (Small) to improve feature extraction capacity and representation learning for subtle vehicle damage patterns. The YOLOv8s architecture contains a larger parameter capacity and deeper feature representation layers, making it more suitable for detecting low-contrast and fine-grained damages such as scratches, dents, and cracks. This configuration retained the targeted hard-class augmentation dataset developed in the previous phase, including the additional 2184 augmented training images generated specifically for the scratch, dent, and crack categories. The training configuration also remained consistent with the previous experiment, using an image size of 640×640 pixels, batch size of 16, 150 training epochs, and early stopping patience of 30 epochs.

Two optimizer configurations were evaluated during this stage:

- YOLOv8s + SGD (Initial learning rate: 0.01, Momentum: 0.937, Weight decay: 0.0005)

- YOLOv8s + AdamW (Initial learning rate: 0.001, Weight decay: 0.01)

The comparison showed that the YOLOv8s architecture produced improved overall detection performance compared with the YOLOv8n-based configurations. In particular, the AdamW configuration achieved slightly higher precision and improved performance on the dent and scratch categories, while the SGD configuration produced marginally better crack detection performance.

Overall, the combination of the YOLOv8s architecture, targeted hard-class augmentation strategy, and AdamW optimization produced the most balanced detection performance across all six vehicle damage categories. Therefore, this configuration was selected as the final damage detection model used in the CrashLens system.

6.5 Functions

The core functionalities of the CrashLens system are listed below.

Commented [15سا]: يحتاج نشيل الكاشن حق التيبيل بس ما ابي الكومننت تروح

Function Name	process_najm_ocr(case_id)
Input	Case id
output	accident number, accident_date, damage_location, extracted_plate, extracted_national_id
Description	This function validates the uploaded Najm accident report by extracting accident details and verifying the national ID and vehicle plate number against the user's stored data. The case status is then updated based on the verification result.
Code Snippet	<pre> async def process_najm_ocr(case_id: str) -> dict: lines = _extract_text_from_pdf_bytes(pdf_bytes) # Existing Najm fields accident_number = _extract_accident_number(lines, pdf_bytes) accident_date = _extract_accident_date(lines) damage_location = _extract_damage_location(lines) # New verification fields from report extracted_plate = _extract_plate_number(lines) extracted_national_id = _extract_national_id(lines) # Matching plate_matched = _match_plate(extracted_plate, expected_plate) national_id_matched = _match_national_id(extracted_national_id, expected_national_id) print(f"[PLATE RAW] extracted={extracted_plate} expected={expected_plate}") print(f"[PLATE NORMALIZED] {f'extracted={_normalize_plate_from_najm(extracted_plate)} 'f'expected={_normalize_plate_from_db(expected_plate)}") print(f"[PLATE MATCH] {plate_matched}") print(f"[NID RAW] extracted={extracted_national_id} expected={expected_national_id}") print(f"[NID MATCH] {national_id_matched}") # OCR completeness required_ocr_ok = all([accident_number, accident_date, damage_location]) # Verification rule: # strong pass = plate + national ID # fallback pass = plate + make + model identity_verified = national_id_matched vehicle_verified = plate_matched and (national_id_matched) verified = required_ocr_ok and vehicle_verified update_data = { "najmReport.accidentNumber": accident_number, "najmReport.accidentDate": accident_date, "najmReport.damageLocation": damage_location, } </pre>

Commented [416]: write it as list of output in natural language

Commented [417R16]: fixed this, but please review

Commented [18سا]: هنا قالت اختصروه واختصرته شيكوا عليه


```

errors = []

if not required_ocr_ok:
    if not accident_number:
        errors.append("accidentNumber missing")
    if not accident_date:
        errors.append("accidentDate missing")
    if not damage_location:
        errors.append("damageLocation missing")

if not plate_matched:
    errors.append("plate number does not match selected vehicle")

if not (national_id_matched ):
    errors.append("report does not sufficiently match selected vehicle / user")

error_message = "Najm verification failed: " + " , ".join(errors)

update_data["status"] = "فشل الفحص"
update_data["ocrError"] = error_message
case_ref.update(update_data)

```

Function Name	<code>process_ocr_registration(file)</code>
Input	Registration Card image
Output	plate_number, make, model, manufacturing_year, color, chassis_number,
Description	This function processes the vehicle registration card image using EasyOCR to extract structured vehicle information. The function returns the extracted data in a structured format.
Code Snippet	<pre> async def process_ocr_registration(file): """ Main function called by FastAPI. Extracts vehicle data from registration card image in Arabic. """ try: print("--- NEW OCR REQUEST STARTED ---") contents = await file.read() nparr = np.frombuffer(contents, np.uint8) image = cv2.imdecode(nparr, cv2.IMREAD_COLOR) if image is None: raise Exception("Invalid image file.") print("1. Running EasyOCR...") result = reader.readtext(image, detail=0, text_threshold=0.5, low_text=0.3) result_detailed = reader.readtext(image, detail=1, text_threshold=0.5, low_text=0.3) print(" -> Done!") normalized_texts = [_normalize_text(t) for t in result] ocr_items = _build_ocr_items(result_detailed) print("2. Extracting fields...") chassis_number = _extract_chassis_number_by_anchor(ocr_items) or _extract_chassis_number(normalized_texts) plate_number = _extract_plate_by_anchor(ocr_items) manufacturing_year = _extract_year_from_roi(image, ocr_items) or _extract_year(normalized_texts) make = _extract_make_by_anchor(ocr_items) or _extract_brand(normalized_texts) model = _extract_model_by_anchor(ocr_items) or _extract_model(normalized_texts) image_width = image.shape[1] color = _extract_color_by_anchor(ocr_items, image_width) or _extract_color(normalized_texts) structured_data = { "plateNumber": plate_number, "make": make, "model": model, "year": manufacturing_year, "color": color, "chassisNumber": chassis_number, } print(f"3. Done! -> {structured_data}") return { "status": "success", "raw_text": normalized_texts, "data": structured_data, } except Exception as e: print(f"!!! SERVER ERROR: {str(e)}") return {"status": "error", "message": f"OCR Error: {str(e)}"} </pre>

Commented [420]: the function name is not meaningful

Commented [419]: I fixed this function, do the same for the above

Function Name	extract_year_from_roi(image, ocr_items: list)
Input	Car registration image as a NumPy array
Output	Manufacturing year
Description	This function extracts the vehicle manufacturing year from the registration card image. It locates the “سنة الصنع” field, crops the nearby region of interest (ROI), enlarges the cropped area to improve OCR accuracy, and applies EasyOCR to detect the year. The function validates the extracted year and returns it as a string. If no valid year is found, an empty string is returned.
Code Snippet	<pre> def _extract_year_from_roi(image, ocr_items: list) -> str: """Extract manufacturing year by cropping near 'سنة الصنع' anchor.""" anchor = _find_anchor_item(ocr_items, ["سنة الصنع", "سنة", "صنع"]) if not anchor: return "" roi = _crop_region(image, anchor["x_min"] - max(anchor["w"] * 2.2, 140), anchor["y_min"] - max(anchor["h"] * 0.4, 10), anchor["x_min"] + max(anchor["w"] * 0.3, 25), anchor["y_max"] + max(anchor["h"] * 1.2, 35)) if roi is None or roi.size == 0: return "" roi = cv2.resize(roi, None, fx=2.0, fy=2.0, interpolation=cv2.INTER_CUBIC) try: roi_result = reader.readtext(roi, detail=0, text_threshold=0.35, low_text=0.2) except Exception: return "" for text in [_normalize_text(t) for t in roi_result]: compact = re.sub(r"\s+", "", text) cands = _extract_valid_year_candidates(compact) if cands: return cands[0] if re.fullmatch(r"\d{4}", compact): rev = _extract_valid_year_candidates(compact[::-1]) if rev: return rev[0] parts = re.findall(r"\d{2}", text) if len(parts) == 2: for c in (parts[0]+parts[1], parts[1]+parts[0]): if _is_valid_year(c): return c return "" </pre>

Link To GitHub Repository: <https://github.com/leen449/2026-GP1-Group7>

Link To Jira: <https://gp1-crashlens.atlassian.net/jira/software/projects/GP/boards/1?atlOrigin=eyJpIjoiMzZkMjRlNDgwNjI1NDcwMzlkODQwMDhkYTA2Yjc2MzliLCJwIjoiajJ9>

7 System Testing

System testing is a critical and important process with the main goal of ensuring that the system meets its user requirements, identifying any confusing aspects for improvement, and checking for errors during system use, this section discuss the experimental results, the user accepting testing, and discussion results.

7.1 Experimental Results

The performance results of the YOLOv8 damage detection experiments are summarized in table 10 below.

The experiments were conducted progressively across multiple stages to evaluate the impact of hyperparameter tuning, augmentation strategies, optimizer selection, and architecture refinement on vehicle damage detection performance.

The baseline YOLOv8n configuration was first evaluated to establish an initial performance reference. The baseline model used an image size of 640×640 pixels, batch size of 16, 100 training epochs, early stopping patience of 20, and the default SGD optimizer. the baseline achieved acceptable performance on visually distinct damage categories, while subtle damage classes such as dent, scratch, and crack remained challenging.

The second experimentation stage focused on general model tuning and augmentation strategies. These experiments included testing multiple image resolutions both 640 and 960 , extending the training duration from 100 to 150 epochs, and applying general training-time augmentation techniques , and multi-scale training strategies to improve robustness against varying object sizes and viewing conditions. improvements were observed in overall precision and mAP values, the difficult damage categories continued to show weaker performance.

The third experimentation stage introduced targeted hard-class augmentation and optimizer adjustment techniques to improve detection performance for scratch, dent, and crack categories. The comparison between SGD and AdamW optimizers under the targeted augmentation setup. The targeted augmentation strategy improved feature representation for the difficult classes and produced more stable detection confidence scores compared with the earlier configurations.

Finally, the fourth experimentation stage evaluated the YOLOv8s architecture using both SGD and AdamW optimizers while retaining the targeted augmentation dataset.

the YOLOv8s model combined with AdamW optimization achieved the best overall performance and was therefore selected as the final damage detection model for CrashLens.

Table 9: Experiments results summary

Configuration	Metric			
	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Config 1 : baseline YOLOv8n	0.7472	0.6799	0.7013	0.5511
Config 2 : extended image size from 640 to 960	0.7647	0.6903	0.7113	0.5496
Config 2 : extended epochs from 100 to 150	0.7566	0.6795	0.7085	0.5538
Config 2 : general augmentation	0.7158	0.7105	0.7061	0.5284
Config 2 : multi scale based augmentation	0.7472	0.6799	0.7013	0.5511
Config 3 :using hard classes augmentation with SGD optimizer	0.7824	0.6836	0.7065	0.5523
Config 3 :using hard classes augmentation with adamW optimizer	0.7431	0.6637	0.6999	0.5434
Config4 : using YOLOV8s with SGD optimizer	0.7842	0.6955	0.7171	0.5646
Config 4 : using YOLOV8s with adamW optimizer	0.8215	0.6476	0.6258	0.5007

7.1.1 Discussion

While the final YOLOv8s model outperformed the baseline across all metrics, improvements on the scratch, dent, and crack classes remained limited. This performance plateau is attributed to the structural characteristics of the CarDD dataset, which presents several well-documented challenges for object detection models. These findings are consistent with those reported in the original CarDD paper [11], where state-of-the-art instance segmentation and object detection models also exhibited notably lower performance on these same three classes. The difficulty stems from three key factors:

- Small object proportion is a significant issue, as small objects account for over 35% of dent instances, 45% of scratch instances, and approximately 90% of crack instances categories that are inherently harder for detection models to localize accurately.
- Shape and scale diversity within these classes is high, as damage size and shape vary unpredictably across different vehicle types and impact scenarios.
- Visual similarity between classes contributes to misclassification, as scratches and cracks share slender linear features and similar colour profiles, causing predictions on one class to negatively affect the detection of the other.

Therefore, the limitations encountered reflect established challenges in automated vehicle damage assessment, particularly around fine-grained damage detection at small scales, and are not unique to the CrashLens implementation.

7.2 User Acceptance Testing

User Acceptance Testing (UAT) represents the final testing stage in the system evaluation process, conducted to verify that CrashLens fulfills its intended business requirements and delivers usable experience for its target audience. The testing assessed the system across several dimensions, including user interface clarity, feature accessibility, and overall ease of use. A total of 8 participants were recruited from the target user base to evaluate the mobile application through a structured questionnaire (refer to [Appendix B](#)). Participants were given access to the CrashLens application and asked to perform many tasks representing the core features, and then answer the questionnaire, which consisted of seven Likert-scale questions regarding the key aspects of the system.

7.2.1 Demographic Of Participants

The UAT participants were selected from the target user population of CrashLens. Demographic information was collected based on age group, gender, and professional background. The participant group included users from multiple age ranges, with balanced representation across the main age categories. Gender distribution was also balanced, with 50% male and 50% female participants. Regarding professional background, all participants identified as everyday car owners or drivers, representing the primary target users of the system.

7.2.2 Questionnaire Results

The UAT questionnaire consisted of seven closed-ended questions using a five-point Likert scale ranging from Strongly Disagree to Strongly Agree. All 8 participants completed all questions (refer to [Appendix C](#)). The questions targeted the following functional areas, which are account registration and login, profile management, manual and OCR-based vehicle addition, accident reporting, vehicle damage image capture, and case status review. After completing these tasks, participants evaluated their experience to assess the usability and effectiveness of the tested features. The results for each question are presented below.

Q1: The process of signing up and logging into the application was straightforward and fast. Responses were highly positive, with 75% of participants selecting Strongly Agree and 25% selecting Agree. No neutral or negative responses were recorded, indicating a high level of user satisfaction with the onboarding and login experience.

Q2: I was able to easily navigate to my profile and successfully update my phone number without confusion. Responses were highly positive, with 87.5% of participants selecting Strongly Agree and 12.5% selecting Agree. No neutral or negative responses were recorded, indicating that participants found the profile navigation and phone number update process clear and easy to use.

Q3: I found the process of adding a new vehicle by manually typing in the details to be clear and user-friendly. Responses were positive overall, with 62.5% of participants selecting Strongly Agree and 37.5% selecting Agree. No neutral or negative responses were recorded, indicating that participants generally found the manual vehicle entry process clear and easy to use.

Q4: Adding a vehicle using OCR was accurate and saved me time compared to manual typing. Responses were generally positive, with 62.5% of participants selecting Strongly Agree and 12.5% selecting Agree, while 25% selected Neutral. The neutral responses may indicate that OCR accuracy or performance varied under certain conditions, such as image quality or registration card clarity, suggesting an area for further improvement.

Q5: The step-by-step process to report a new accident and submit a case felt logical and easy to follow. Responses were positive overall, with 62.5% of participants selecting Strongly Agree and 37.5% selecting Agree. No neutral or negative responses were recorded, indicating that participants found the accident reporting and case submission workflow clear, organized, and easy to follow.

Q6: The camera interface (with real-time blur and lighting checks) helped me take better photos of the vehicle damage. Most participants expressed positive feedback toward the camera assistance features, with 62.5% selecting Strongly Agree and 25% selecting Agree, while 12.5% selected Neutral. The neutral response may indicate that some participants were already comfortable taking vehicle photos independently or did not perceive a significant difference from the provided guidance features.

Q7: When viewing the Case Details screen, the information provided (e.g., status badges, damage annotations) was clear and easy to understand. Participants provided consistently positive feedback, with 75% selecting Strongly Agree and 25% selecting Agree. No neutral or negative responses were recorded, indicating that the case details and AI-generated information were presented clearly and effectively to users.

7.2.3 Discussion

Overall, the UAT results demonstrate a high level of user satisfaction with the CrashLens system across all evaluated functional areas. Most questions received highly positive responses, with the majority of participants selecting either Strongly Agree or Agree across the evaluated features. A small proportion of neutral responses was recorded in the OCR-based vehicle addition and camera guidance questions, indicating minor areas for potential improvement.

The strongest performing areas were profile management and the account registration flow, both of which achieved very high Strongly Agree responses. This suggests that the authentication, onboarding, and profile navigation experiences were clear, intuitive, and easy for users to complete successfully. Similarly, the accident reporting and case submission workflow received consistently positive feedback, indicating that users found the step-by-step process logical and easy to follow.

The OCR-based vehicle addition feature was positively received overall; however, some participants selected Neutral regarding its accuracy and time-saving capabilities. This may be attributed to variations in registration card quality, image clarity, or lighting conditions during testing, which can affect OCR extraction accuracy. Future improvements could focus on enhancing image preprocessing techniques and providing additional guidance during the scanning process to improve reliability.

The camera guidance feature also recorded a small number of neutral responses. This may indicate that some participants did not consistently notice the real-time blur and lighting assistance during image capture. Future iterations of the system could improve the visibility of guidance indicators or provide more interactive feedback mechanisms to further assist users while capturing vehicle damage images.

The Case Details screen and damage annotation presentation performed particularly well, receiving exclusively positive feedback from participants. Users were able to clearly understand the provided information, including status indicators and AI-generated damage annotations. This suggests that the visual presentation and organization of case-related information were effective and user-friendly.

8 Conclusion And Future Work

8.1 Summary

CrashLens was developed as a mobile-first vehicle damage assessment platform targeting individual car owners within the Saudi Arabian market. The system addresses a clear gap in the existing landscape, where enterprise-focused solutions such as Tractable AI, Inspektlabs, and RavinAI serve large insurance institutions but remain inaccessible to everyday drivers. By combining automated AI-based damage detection, OCR-powered vehicle registration extraction, guided image capture, structured case submission, and an administrative review workflow, CrashLens delivers a comprehensive and localized solution designed around the real-world needs of the vehicle damage assessment ecosystem.

The development process was grounded in a thorough requirements elicitation phase conducted through semi-structured interviews with professional vehicle damage assessors at Taqdeer certified centers. These interviews provided direct insight into current assessment workflows, cost estimation methodologies, and operational challenges, ensuring that the system requirements were practical, realistic, and aligned with industry standards. The system was subsequently designed, implemented, and evaluated through both experimental model testing and user acceptance testing, yielding results that validate the system's core functionality and usability.

8.2 Global and Local Impact

Locally, CrashLens directly addresses inefficiencies in the vehicle damage assessment process by enabling individual car owners to initiate and track damage assessment cases digitally, without the need to physically visit an assessment center for every stage of the process. This is particularly significant in the context of Saudi Arabia's Vision 2030 agenda, which emphasizes digital transformation across public and private sectors. By reducing reliance on manual, paper-based workflows and empowering citizens with a transparent and accessible digital tool, CrashLens contributes meaningfully to the modernization of the insurance and vehicle assessment sector within the Kingdom.

Globally, the system contributes to the growing body of applied research in automated vehicle damage assessment. The challenges encountered during model training particularly around

detecting subtle damage types such as scratches, dents, and cracks in the CarDD dataset reflect and reinforce findings reported in academic literature. CrashLens demonstrates that even with dataset limitations, a carefully iterative training approach can yield practically deployable models, and its architecture may serve as a reference point for similar localized damage assessment applications in other markets.

8.3 Problems and Challenges Encountered

Several challenges were encountered during the development of CrashLens. One of the main challenges was achieving reliable detection performance for subtle vehicle damage types such as scratches, dents, and cracks, particularly due to dataset limitations and variations in vehicle designs, image quality, and lighting conditions. Another challenge involved verifying Najm accident reports early in the submission process. To improve user experience, the workflow was redesigned so that OCR extraction and validation of accident information are performed immediately after upload, allowing invalid reports to be detected before case submission is completed. Moreover, extracting structured information from vehicle registration cards also presented challenges due to the complexity of Arabic text and the reflective design of the card, which sometimes caused lighting reflections that affected OCR extraction accuracy. These challenges were addressed using an anchor-based extraction approach and image preprocessing techniques for small fields such as the manufacturing year. Finally, providing reliable real-time image quality guidance during damage photo capture was challenging. Therefore, the initial sharpness detection method was replaced with a more robust FFT-based approach to improve the consistency of blur detection and image quality feedback.

8.4 Limitations of the System

CrashLens is limited to the assessment of externally visible damage as with all vision-based damage assessment systems. Internal mechanical or structural damage that is not visually apparent from exterior photographs cannot be detected or reported through the current system.

8.5 Main Contribution of the Project

The main contributions of this project include implementing OCR-based extraction for vehicle registration cards, implementing OCR-based verification for Najm accident reports, developing a vehicle damage detection model, and integrating real-time image capture guidance to improve the quality of captured vehicle damage images. In addition, the project contributes by developing a mobile-first vehicle damage assessment application tailored to the Saudi market.

8.6 Future Work

Future work for CrashLens will focus on the implementation of a repair cost estimation component in addition to damage severity estimation capable of generating realistic and structured predictions based on detected vehicle damage and vehicle-related information.

As well as an expanded administrative dashboard to support case monitoring, objection review, and overall system management through a centralized interface for administrators.

Future work will also focus on improving vehicle damage detection performance by expanding the training dataset using locally sourced vehicle damage images collected from real-world Saudi road and insurance scenarios. This is expected to improve model robustness, localization accuracy, and adaptation to local vehicle damage patterns and environmental conditions.

Commented [421]: Need rewriting: just describe future works as what you will do in release 2 features in addition to improve the model performance using local datasets

9 References

- [1] "ر. ف. ا. الشيخ، "نظم المعلومات الجغرافية كأداة في تحليل الحوادث المرورية and ح. ع. الرفاعي Ajsp, 2022. Accessed: Jan. 16, 2026. [Online]. Available: <https://www.ajsp.net/research/%D9%86%D8%B8%D9%85%20%D8%A7%D9%84%D9%85%D8%B9%D9%84%D9%88%D9%85%D8%A7%D8%AA%20%D8%A7%D9%84%D8%AC%D8%BA%D8%B1%D8%A7%D9%81%D9%8A%D8%A9%20%D9%83%D8%A3%D8%AF%D8%A9%20%D9%81%D9%8A%20%D8%AA%D8%AD%D9%84%D9%8A%D9%84%20%D8%A7%D9%84%D8%AD%D9%88%D8%A7%D8%AF%D8%AB%20%D8%A7%D9%84%D9%85%D8%B1%D9%88%D8%B1%D9%8A%D8%A9.pdf>
- [2] "معايينة درجة خطورة حوادث المرور بالمملكة العربية السعودية ومقارنتها ببعض الدول الأخرى" Accessed: May 09, 2026. [Online]. Available: <https://journals.ku.edu.kw/jgaps/index.php/jgaps/article/view/1699/1367>
- [3] ع. ا. ر. علي، "التعريف بحادث السير ومفهومه في المملكة العربية السعودية"، *مجلة العلوم الاقتصادية و الإدارية و القانونية*, vol. 5, no. 12, pp. 63–46, Jun. 2021, doi: 10.26389/AJSRP.B291020.
- [4] N. Rahul, "Vehicle and Property Loss Assessment with AI: Automating Damage Estimations in Claims," *International Journal of Emerging Research in Engineering and Technology*, vol. 1, no. 4, pp. 38–46, Dec. 2020, doi: 10.63282/3050-922X.IJERET-V114P105.
- [5] "Saudi Vision 2030." Accessed: May 09, 2026. [Online]. Available: https://www.vision2030.gov.sa/media/rc0b5oy1/saudi_vision203.pdf
- [6] "Introduction to Computer Vision and Image Processing," Coursera. Accessed: May 09, 2026. [Online]. Available: <https://www.coursera.org/learn/introduction-computer-vision-watson-opencv>
- [7] A. Singh, "Car Damage Assessment: Here's Everything You Need to Know," Spyne. Accessed: May 09, 2026. [Online]. Available: <https://www.spyne.ai/blogs/car-damage-assessment>
- [8] "Taqdeer." Accessed: May 09, 2026. [Online]. Available: <http://taqdeer.sa/>
- [9] "Professional Standards for Vehicle Damage Assessment." Accessed: May 09, 2026. [Online]. Available: <https://taqdeer.sa/media/ilcjm1sr/professional-standards.pdf>
- [10] J. Qaddour and S. A. Siddiqua, "Automatic damaged vehicle estimator using enhanced deep learning algorithm," *Intelligent Systems with Applications*, vol. 18, p. 200192, May 2023, doi: 10.1016/j.iswa.2023.200192.
- [11] "Taqdeer Damage Assessment Report | PDF | Business | Economics," Scribd. Accessed: May 09, 2026. [Online]. Available: <https://www.scribd.com/document/511609952/TAQDEER-CENTER-ESTIMATE>
- [12] "Demystifying Machine Learning: A Comprehensive Guide." Accessed: May 09, 2026. [Online]. Available: <https://www.oracle.com/sa/artificial-intelligence/machine-learning/what-is-machine-learning/>
- [13] D. Bergmann, "What Is Deep Learning? | IBM." Accessed: May 09, 2026. [Online]. Available: <https://www.ibm.com/think/topics/deep-learning>

- [14] S. Fernando, "From the Human Brain to the Perceptron," Data Science Collective. Accessed: May 09, 2026. [Online]. Available: <https://medium.com/data-science-collective/from-the-human-brain-to-the-perceptron-87e24c5aae0d>
- [15] "YOLO Algorithm for Object Detection Explained [+Examples]." Accessed: May 09, 2026. [Online]. Available: <https://www.v7darwin.com/blog/yolo-object-detection>
- [16] B. Academy, "Scan the World Using Object Detection," Medium. Accessed: May 09, 2026. [Online]. Available: <https://medium.com/@comeblossom.gh/scan-the-world-using-object-detection-d3efbabb7ab0>
- [17] "What Is Transfer Learning? [Examples & Newbie-Friendly Guide]." Accessed: May 09, 2026. [Online]. Available: <https://www.v7darwin.com/blog/transfer-learning-guide>
- [18] "Fine-tuning LLMs Guide | Unsloth Documentation." Accessed: May 09, 2026. [Online]. Available: <https://unsloth.ai/docs/get-started/fine-tuning-llms-guide>
- [19] "The Power of Computer Vision in AI: Unlocking the Future!," Simplilearn.com. Accessed: May 09, 2026. [Online]. Available: <https://www.simplilearn.com/computer-vision-article>
- [20] R. D. C. Stryker Cole, "What Is Computer Vision? | IBM." Accessed: May 09, 2026. [Online]. Available: <https://www.ibm.com/think/topics/computer-vision>
- [21] H. Vedoveli, "Metrics Matter: A Deep Dive into Object Detection Evaluation," Medium. Accessed: May 09, 2026. [Online]. Available: <https://medium.com/@henriquevedoveli/metrics-matter-a-deep-dive-into-object-detection-evaluation-ef01385ec62>
- [22] "Key Object Detection Metrics for Computer Vision," Roboflow Blog. Accessed: May 09, 2026. [Online]. Available: <https://blog.roboflow.com/object-detection-metrics/>
- [23] Q. Zhang, X. Chang, and S. B. Bian, "Vehicle-Damage-Detection Segmentation Algorithm Based on Improved Mask RCNN," *IEEE Access*, vol. 8, pp. 6997–7004, 2020, doi: 10.1109/ACCESS.2020.2964055.
- [24] H. Jin, X. Wang, and Z. Wu, "An Anchor Free Car Damage Detection Method," in *Proceedings of the 2023 9th International Conference on Computing and Data Engineering*, in ICCDE '23. New York, NY, USA: Association for Computing Machinery, Jun. 2023, pp. 66–71. doi: 10.1145/3589845.3589848.
- [25] M. J. Hasan, C. K. Nguyen, Y. L. Boo, H. Jahani, and K.-L. Ong, "Vehicle Damage Detection Using Artificial Intelligence: A Systematic Literature Review", doi: 10.1002/widm.70027.
- [26] D. Mallios, L. Xiaofei, N. McLaughlin, J. M. D. Rincon, C. Galbraith, and R. Garland, "Vehicle Damage Severity Estimation for Insurance Operations Using In-The-Wild Mobile Images," *IEEE Access*, vol. 11, pp. 78644–78655, 2023, doi: 10.1109/ACCESS.2023.3299223.
- [27] A. S. Elbhrawy, M. A. Belal, and M. S. Hassanein, "CES: Cost Estimation System for Enhancing the Processing of Car Insurance Claims," *Journal of Computing and Communication*, Jan. 2024, doi: 10.21608/jocc.2024.339922.
- [28] "Home," Tractable. Accessed: May 09, 2026. [Online]. Available: <https://tractable.ai/>

- [29] “AI Vehicle Inspection Software | Automated Platform | Inspektlabs.” Accessed: May 09, 2026. [Online]. Available: <https://inspektlabs.com/>
- [30] “Ravin AI | Transforming Auto Damage Assessments and Claims for the Digital Age.” Accessed: May 09, 2026. [Online]. Available: <https://www.ravin.ai/>

APPENDIX A: Interviews

INTERVIEW 1:

The first interview was conducted with a professional vehicle damage assessor employed at a certified Taqdeer assessment center. The interview was carried out in a semi-structured format, allowing the participant to speak freely about their daily workflows and professional experiences. The session covered ten key topics related to the vehicle damage assessment process, cost estimation methodology, operational challenges, and potential system improvements. The objective was to gain firsthand insight into real-world assessment practices that would inform the functional and non-functional requirements of CrashLens.

Q1: What is the current damage assessment procedure?

the process begins at the reception counter, where vehicle data is registered in the system, including the accident number and vehicle registration details. Once registered, the vehicle proceeds to the assessment lane. The assessor conducts a standardized inspection that includes capturing photographs from all angles of the vehicle, photographing visible damage areas, documenting the seatbelt to verify its functionality, capturing the dashboard and odometer, and recording the chassis number. This documentation process is mandatory for all cases and ensures consistency across assessments.

Q2: How are hidden or internal damages handled?

only visible damage is evaluated during the initial inspection. Internal damage is not examined at this stage because dismantling the vehicle is not performed at the assessment center. If internal damage is later discovered at a workshop, the customer may submit an objection or additional objection. Supporting photos must be provided, and in some cases, another assessor may be assigned to review the additional damage. The system does not approve part replacements without documented evidence.

Q3: How is repair cost calculated after inspection?

repair cost calculation depends on whether the damaged component requires replacement or repair. In cases requiring replacement, the damaged parts are recorded, labor cost is determined by the assessor, and the request is sent to a contracted supplier who provides the spare part price.

The total cost is then communicated to the client. If the part requires repair only, the assessor calculates an average repair cost based on market standards.

Q4: What factors affect repair cost?

Several factors influence the final repair cost. These include the type of paint used on the vehicle, as certain finishes such as pearlescent or metallic paints increase cost. The vehicle's age is also significant; vehicles older than ten years may be handled under a different evaluation mechanism. Additionally, whether spare parts are original or commercial affects pricing, while labor cost generally remains standardized.

Q5: How are spare part prices updated?

Spare part prices are determined through contracted suppliers affiliated with the system. Approximately 20–30 suppliers are involved. The suppliers are responsible for setting and updating pricing, and they coordinate with authorized dealerships. The assessment authority does not directly determine spare part prices.

Q6: What challenges do assessors face?

identifying chassis damage is a critical challenge. Assessing mechanical components and determining whether damage is new or pre-existing can also be difficult. In some situations, the assessor must explain to customers that certain damages are unrelated to the current accident. identifying chassis damage is a critical challenge. Assessing mechanical components and determining whether damage is new or pre-existing can also be difficult. In some situations, the assessor must explain to customers that certain damages are unrelated to the current accident.

Q7: What is the average time required for assessment?

Minor damage cases typically take between 10 to 15 minutes. However, major damage cases may require extended review and, in some instances, committee evaluation that can take one to two days.

Q8: What types of damage are most difficult to evaluate?

Electrical damage and mechanical damage were identified as the most difficult to evaluate, especially when multiple components are involved. There is also a risk of overlooking hidden or interconnected damage.

Q9: Are there any suggested improvements?

the system has significantly improved over time and currently operates effectively. No major improvements were suggested.

Q10: How was electronic assessment implemented during Hajj?

During Hajj season, the centre operated continuously due to increased case volume. Electronic and remote assessment was primarily used for minor and clearly visible damage. Major collisions were not evaluated remotely and required in-person assessment or referral to a market valuation expert.

INTERVIEW 2:

The second interview was conducted with a professional vehicle damage assessor working within the Taqdeer electronic assessment system. The interview followed a semi-structured format, allowing the participant to elaborate on their experience with the fully digitized assessment workflow. The session focused on understanding how the current electronic system operates, how costs are calculated, how fraud is detected, and what challenges assessors encounter in practice. The insights gathered contributed directly to the design of CrashLens's guided image capture, cost estimation, and administrative review features.

Q1: What is the current mechanism used for vehicle damage assessment?

The current mechanism is fully electronic and operates through the Taqdeer system. Upon vehicle arrival, the case is registered digitally and assigned to an assessor. The system guides the assessor regarding required photographs, including general vehicle images, damage areas, structural components, license plate, odometer, airbags, and seatbelts. All damaged parts are recorded in the system, after which labor cost is determined by the assessor and spare part pricing is requested from suppliers.

Q2: How are hidden damages handled?

Internal damages are not identified during the initial assessment. The vehicle must undergo workshop dismantling to detect such damage. If additional issues are discovered, a supplementary objection can be submitted along with supporting images. An assessor may review the updated information before approval.

Q3: How is repair cost calculated?

Repair cost consists of supplier-determined spare part pricing and assessor-determined labour cost. If prior compensation has been issued, the system adjusts compensation accordingly. Additional labour charges are applied if further damage is discovered after dismantling.

Q4: What are the main factors affecting repair cost?

Repair cost is influenced by whether parts are original or commercial, previous repair history, paint type, and whether specialized programming is required for luxury vehicles. Vehicle age may also influence evaluation method.

Q5: How are spare parts priced and updated?

Spare parts are priced by suppliers in coordination with authorized dealers. Pricing responsibility lies entirely with suppliers, not with the assessment authority.

Q6: What challenges do assessors face?

Major challenges include deployed airbags, damaged onboard computer systems, extensive structural damage, and cases requiring underbody dismantling. Diagnostic tools may be required to detect full damage extent.

Q7: How long does assessment take?

Assessment duration varies. Minor cases take approximately 10–15 minutes, moderate cases up to 30 minutes, and heavily damaged vehicles require longer evaluation.

Q8: Which types of damage are hardest to evaluate?

Severe and heavily impacted damages, particularly structural or electronic system damage, are the most difficult.

Q9: What improvements are needed?

I am satisfied with the current system and do not recommend specific improvements.

Q10: How was electronic assessment implemented during Hajj?

The electronic system was applied in certain regions during Hajj, primarily for minor damage cases. It helped manage high case volumes efficiently.

INTERVIEW 3:

The third interview was conducted with a professional vehicle damage assessor experienced in working with both the Najm and Taqdeer systems. The interview was conducted in a semi-structured format and covered the full assessment lifecycle, from vehicle arrival and documentation through to cost estimation, total loss evaluation, and fraud detection. The participant also provided insight into suggested system improvements and the use of remote electronic assessment during peak seasons. The findings from this interview further validated the requirements identified in the previous two interviews and introduced additional considerations around total loss thresholds and system integration.

Q1: What is the current damage assessment procedure?

the process begins by verifying the accident number, whether issued by Traffic Police or Najm, along with confirming the vehicle registration and customer contact information. Once the vehicle is registered, the assessor conducts a full 360-degree inspection. This includes documenting visible damage through photographs and verifying key vehicle information. The assessor ensures that all required documentation is captured before proceeding with evaluation. The case is then processed within the Najm and Taqdeer systems, and repair costs are calculated based on average workshop market rates rather than dealership pricing, even for newly manufactured vehicles.

Q2: How are hidden or internal damages handled?

only visible damages are assessed during the initial inspection. If hidden or internal damage is suspected, further inspection is required. In such cases, components may need to be dismantled at a workshop to determine the full extent of the damage. Any additional damage discovered after dismantling must be documented and submitted for supplementary review. Approval for additional damage may require supporting evidence and further evaluation.

Q3: How is the repair cost calculated after inspection?

Repair cost is calculated based on several structured criteria. The assessor first classifies the severity of damage as minor, medium, or major. Spare part prices are obtained from approved suppliers affiliated with the Saudi Authority for Accredited Valuers, while labour cost is determined based on average workshop rates. Vehicle category and paint specifications are also considered. Dealership pricing is not used in the calculation. If the repair cost exceeds a defined threshold, such as 50% of the vehicle's market value, the vehicle may be evaluated for total loss.

Q4: What factors affect repair cost?

repair cost is influenced by the severity of damage, the type of damaged components, spare part pricing, and labor cost averages. The vehicle's age plays a role, particularly for vehicles older than ten years. Structural or safety-related damage, such as chassis impact, airbag deployment, or engine damage, significantly increases cost. Additionally, if parts were previously repaired or compensated for, this affects the evaluation outcome.

Q5: How are spare part prices updated?

Spare part prices are provided by approved suppliers affiliated with the Saudi Authority for Accredited Valuers. Pricing is based on the specific part name and the vehicle's chassis number to ensure accuracy. Suppliers coordinate with authorized dealerships when necessary, and they are responsible for maintaining updated and accurate pricing. The assessment authority does not directly set spare part prices.

Q6: What challenges do assessors face?

Major collisions involving structural damage are difficult to evaluate. Deployed airbags, engine damage, and damage affecting safety systems require careful inspection. Cases involving extensive dismantling or hidden damage extension also increase complexity. Additionally, ensuring accurate documentation and detecting potential fraud are ongoing challenges. Dependency on external systems such as Najm reports can also affect workflow efficiency.

Q7: What is the average time required for assessment?

The average inspection time is approximately 15 minutes for standard cases. However, assessment duration varies depending on damage severity. Major or complex cases may require extended evaluation time.

Q8: What are the most difficult types of damage?

The most difficult cases involve severe structural damage, chassis impact, airbag deployment, engine damage, and damage affecting safety-related systems. Total loss evaluations are also complex, particularly when multiple criteria must be assessed. A vehicle may be declared a total loss if repair cost exceeds 50% of its value, if it is older than ten years, if there is no official dealer in Saudi Arabia, or if it is classified as a classic or inherited vehicle.

Q9: Are there any suggested improvements?

the authority continuously updates and improves the system. Suggestions for enhancement included strengthening fraud detection alerts, improving integration between Najm and Bashar systems, enhancing electronic documentation features, and ensuring standardized pricing accuracy across suppliers.

Q10: How was electronic assessment implemented during Hajj season?

During Hajj season, electronic assessment through Najm's remote estimation feature was primarily used for minor damage cases. Remote estimation covered labor costs only and was not applicable for medium or major damage cases. If Najm could not assess the case remotely, it was transferred to the Traffic Department for further processing. The electronic system contributed to managing the high case volume during peak seasonal periods.

APPENDIX B: User Acceptance Testing Questionnaire

The following questionnaire was distributed to UAT participants to evaluate the usability and functionality of the CrashLens mobile application. Participants were asked to interact with the application before completing the questionnaire.

Section 1 — Demographics

1. What is your age group?
 - 18–24
 - 25–34
 - 35–44
 - 45–54
 - 55 or older
2. What is your gender?
 - Female
 - Male
 - Prefer not to say
3. Which of the following best describes your role or background?
 - Everyday car owner / driver
 - Insurance company employee / administrator
 - Auto repair shop worker / manager
 - Other

Section 2 — System Usability

For each statement below, please indicate your level of agreement using the following scale:

Strongly Agree — Agree — Neutral — Disagree — Strongly Disagree

4. The process of signing up and logging into the application was straightforward and fast.

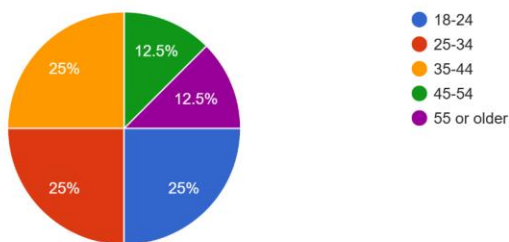
5. I was able to easily navigate to my profile and successfully update my phone number without confusion.
6. I found the process of adding a new vehicle by manually typing in the details to be clear and user-friendly.
7. Adding a vehicle using OCR was accurate and saved me time compared to manual typing.
8. The step-by-step process to report a new accident and submit a case felt logical and easy to follow.
9. The camera interface (with real-time blur and lighting checks) helped me take better photos of the vehicle damage.
10. When viewing the Case Details screen, the information provided (e.g., status badges, damage annotations) was clear and easy to understand.

APPENDIX C: UAT Questionnaire Responses

Section 1-Demographics responses

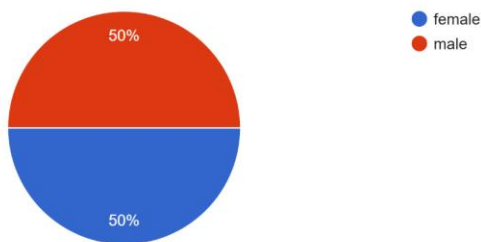
What is your age group?

8 responses



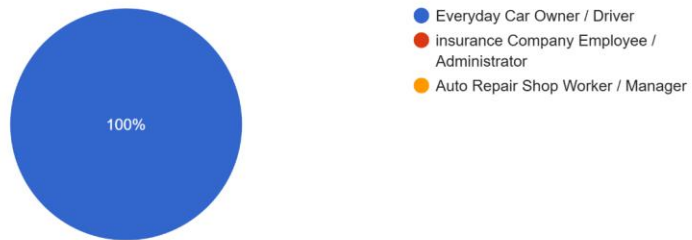
what's your gender?

8 responses



Which of the following best describes your role or background?

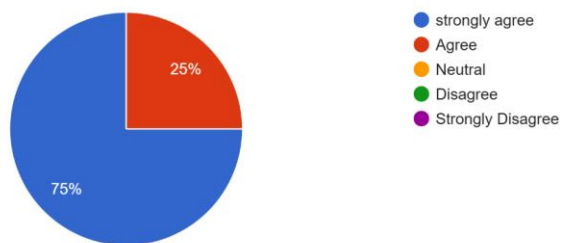
8 responses



Section 2 —System Usability responses

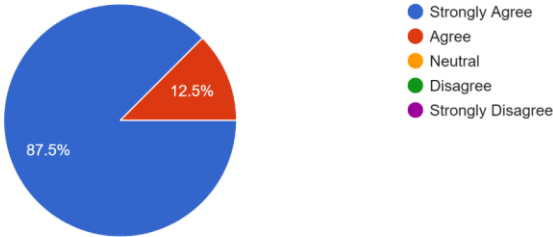
The process of signing up and logging into the application was straightforward and fast.

8 responses



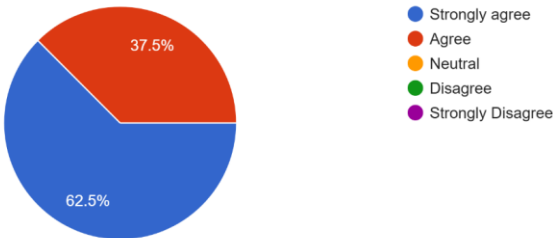
I was able to easily navigate to my profile and successfully update my phone number without confusion.

8 responses

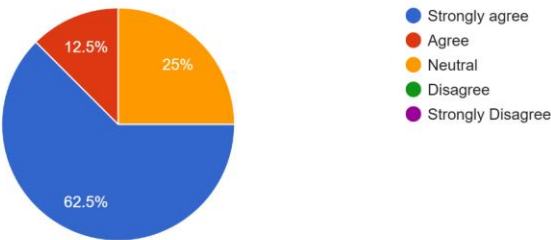


I found the process of adding a new vehicle by manually typing in the details to be clear and user-friendly.

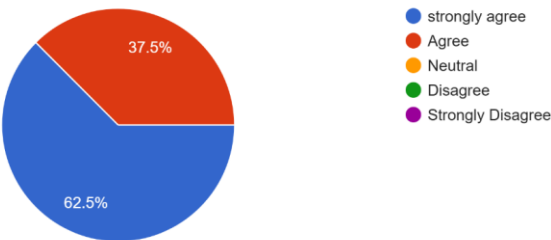
8 responses



adding vehicle using OCR was accurate and saved me time compared to manual typing.
8 responses

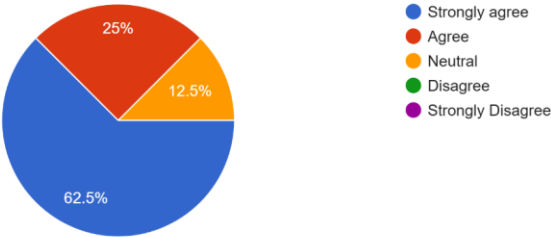


The step-by-step process to report a new accident and submit a case felt logical and easy to follow.
8 responses



The camera interface (with real-time blur and lighting checks) helped me take better photos of the vehicle damage.

8 responses



When viewing the Case Details screen, the information provided (e.g., status badges, damage annotations) was clear and easy to understand.

8 responses

